

DSA Cheatsheet — Tackle Every Topic With High Confidence

144 problems across 29 patterns, grouped into 4 tiers. For each topic: the mental model that makes you *recognize* the pattern, the concrete skeleton to *execute* it, the complexity, and the single trap that *sinks* most attempts.

Table of Contents

- **Orientation:** [How to use this](#) · Pattern Selection · [Tier Mindsets](#) · Cross-Cutting Disambiguation
- **Tier 1 — Foundation:** [bfs](#) · [fast_slow_pointers](#) · [hashmap](#) · [merge_intervals](#) · [sliding_window](#) · [string](#) · [two_pointers](#)
- **Tier 2 — Intermediate:** [binary_search](#) · [bit_manipulation](#) · [design](#) · [dfs](#) · [divide_and_conquer](#) · [dynamic_programming](#) · [math](#) · [prefix_sum](#) · [randomized](#) · [stack](#) · [top_k_elements](#) · [two_heaps](#)
- **Tier 3 — Advanced:** [backtracking](#) · [cyclic_sort](#) · [modified_binary_search](#) · [subsets](#) · [trie](#)
- **Tier 4 — Expert:** [graph](#) · [greedy](#) · [matrix_traversal](#) · [monotonic_stack](#) · [union_find](#)

How to use this

- **Stuck on a new problem?** → Scan the **Pattern Selection** table; match the problem's signals to a pattern.
- **Learning a topic?** → Read its Mindset → When → Skeleton → Gotcha, then drill every problem in its table.
- **Tiers build on each other:** T1 (pointers / windows / hash) → T2 (heaps / DP / binary search) → T3 (combinatorial search) → T4 (advanced structures & invariants). Don't skip ahead — Tier 2+ problems are usually a Tier 1 pattern wrapped in extra state.

Pattern Selection (signal → pattern)

Signal in the problem statement	Use this pattern
sorted array + pair/triplet summing to target	two_pointers
longest / shortest / count of	sliding_window
subarrays/substrings under a constraint	
[start,end] ranges, meetings, coverage, timeline	merge_intervals
shortest path / min steps on an unweighted grid or graph	bfs
linked list cycle / middle / kth-from-end, O(1) space	fast_slow_pointers
count / frequency / membership / O(1) lookup / dedupe	hashmap
reformat / parse / case-validate strings	string
sorted / rotated / monotonic array, or O(log n) required	binary_search / modified_binary_search

Signal in the problem statement	Use this pattern
“minimize the max” / “maximize the min” + a feasibility test	<code>modified_binary_search</code> (answer-space)
return all combinations / permutations / subsets	<code>subsets</code> / <code>backtracking</code>
find a valid arrangement satisfying a constraint (prune hard)	<code>backtracking</code>
kth largest/smallest / top-k / k-closest	<code>top_k_elements</code>
median of a stream / sliding window	<code>two_heaps</code>
matching / nesting / decode (brackets, run-length)	<code>stack</code>
“next greater/smaller”, largest rectangle, subarray contribution	<code>monotonic_stack</code>
max/min ways , optimal substructure, overlapping subproblems	<code>dynamic_programming</code>
subarray sum/product = k , “equal # of X and Y”, range queries	<code>prefix_sum</code>
dependencies / prerequisites / valid ordering	<code>graph</code> (topological sort)
undirected “are these connected?” / redundant edge / same-set	<code>union_find</code>
prefix matching / autocomplete / word dictionary / wildcard	<code>trie</code>
numbers in range [1,n] (or [0,n]), find missing/duplicate, $O(1)$ space	<code>cyclic_sort</code>
“every element appears twice except one”, parity, bitmask	<code>bit_manipulation</code>
design a class with $O(1)$ ops / eviction policy	<code>design</code>
implement randX from randY , uniform sampling	<code>randomized</code>
matrix rotate/spiral/diagonal, or “nearest X per cell”	<code>matrix_traversal</code>
reachability / can-finish / “find the special node”	<code>graph</code> (degree arithmetic)
split → recurse → merge; counting inversions	<code>divide_and_conquer</code>

Tier Mindsets (the 30-second orientation)

- **Tier 1 — Foundation:** *Two cursors over a linear structure.* Pointers slide, windows expand/contract, hashes give $O(1)$ recall. ~70% of interview questions are a Tier 1 pattern in disguise. **Goal:** make pointer movement and hash-map lookups muscle memory.
- **Tier 2 — Intermediate:** *Add structure & state.* Heaps impose ordering, DP memoizes state, binary search halves a monotone space, trees/graphs need recursion + a visited set, design layers $O(1)$ structures. **Goal:** know *when* to invoke each — the hard part is recognition, not the mechanics.
- **Tier 3 — Advanced:** *Search a combinatorial space smartly.* Backtrack with pruning, enumerate with subsets, walk a trie, use the array as its own hash (cyclic sort), binary-search the *answer* not the array. **Goal:** disciplined choose→explore→un-choose, and recognizing feasibility-monotonic search.
- **Tier 4 — Expert:** *Advanced structures & invariants.* Union-find for dynamic connectivity, monotonic stacks for bounded-extremum problems, greedy proofs, topological order, matrix index-gymnastics. **Goal:** know the right invariant and when a greedy choice is provably safe vs when you must fall back to DP/multi-pass.

Tier 1 — Foundation

bfs

- **Mindset:** BFS explores outward in concentric “distance rings” — the first time you reach a node is via the shortest path. *The* tool for unweighted shortest-path and level-by-level work. Think of it as a wavefront spreading from sources.
- **When to use:** “shortest path / min steps / min minutes”, “level by level”, spread/radiate, nearest/closest, multi-source propagation, tree row operations.
- **Skeleton:**

```
# --- Single-source BFS (shortest path) ---
from collections import deque
queue = deque([(start, 0)])      # (node, dist) or (r, c, dist)
visited = {start}               # mark on ENQUEUE, not dequeue
while queue:
    node, dist = queue.popleft()
    if node == target:
        return dist
    for nb in neighbors(node):
        if nb not in visited:
            visited.add(nb)      # ← mark HERE before append
            queue.append((nb, dist + 1))

# --- Level-by-level BFS (tree row operations) ---
queue = deque([root])
while queue:
    level_size = len(queue)     # snapshot level boundary
    level_result = []
    for _ in range(level_size):
        node = queue.popleft()
        level_result.append(node.val)
        if node.left: queue.append(node.left)
        if node.right: queue.append(node.right)
    # process level_result here

# --- Multi-source BFS (p994 pattern) ---
queue = deque()
fresh = 0
for r, c in all_cells:
    if cell == SOURCE: queue.append((r, c)) # seed ALL sources at t=0
    elif cell == FRESH: fresh += 1
if fresh == 0: return 0        # early exit: nothing to spread to
minutes = 0
while queue:
    for _ in range(len(queue)): # process one wave
        r, c = queue.popleft()
        for dr, dc in directions:
```

```

nr, nc = r+dr, c+dc
if in_bounds and grid[nr][nc] == FRESH:
    grid[nr][nc] = ROTTEN # mark on enqueue (mutate grid)
    fresh -= 1
    queue.append((nr, nc))
    if queue: minutes += 1 # ← only count if wave produced work
return minutes if fresh == 0 else -1

```

- **Complexity:** $O(V+E)$ time, $O(V)$ space (queue holds at most all nodes).
- **Killer gotcha:** Mark visited **on enqueue**, never on dequeue — else the same cell gets re-enqueued multiple times → TLE. In grid problems, mutate the cell value directly (e.g., `grid[nr][nc] = 1`) to avoid a separate visited set. For p994, increment `minutes` only **if queue:** after each wave — avoids off-by-one from the final empty round. p1091: returns -1 if `grid[0][0]==1` OR `grid[n-1][n-1]==1`; also handles `n==1` → `return 1`.

Problem	Essence	Key Trick
p102 Level Order	Tree BFS → list-of-lists	<code>level_size = len(queue)</code> snapshots each row boundary
p449 Serialize/Deserialize BST	Two valid approaches	BFS Codec: level-order with "null" markers for both encode/decode. DFS Codec (compact): pre-order serialize (no nulls) + BST-bounds recursive rebuild (<code>build(lo, hi)</code>)
p463 Island Perimeter	BFS or $O(1)$ math	BFS: per cell, each OOB/water neighbor → +1. $O(1)$: <code>land*4 - 2*shared_edges</code> (scan right+down pairs only)
p513 Bottom Left Tree Value	Left-to-right BFS	Capture <code>queue[0].val</code> at the start of each level; last captured value is bottom-left (no reversal needed)
p515 Largest Value Each Row	Level-order, track max	<code>max_val = float("-inf")</code> , updated per node in level loop
p994 Rotting Oranges	Multi-source BFS	Seed all 2s at <code>t=0</code> ; count fresh upfront; if queue: <code>minutes += 1</code> after each wave; return -1 if <code>fresh > 0</code>
p1091 Shortest Path Binary Matrix	Single-source BFS, 8 dirs	Mark <code>grid[r][c] = 1</code> on enqueue; early return when neighbor == <code>(n-1,n-1)</code> ; <code>n==1</code> → <code>return 1</code> ; blocked start/end → -1

fast_slow_pointers

- **Mindset:** Two pointers moving at different speeds through a sequence — if a cycle exists, fast eventually laps slow and they meet; if no cycle, fast exhausts the sequence and slow sits at the midpoint.
- **When to use:**

- Detect a cycle in a linked list ($O(1)$ space vs hash-set $O(N)$)
- Find the middle of a linked list
- Apply Floyd's algorithm to an *abstract* sequence defined by a `next(state)→state` function (e.g. happy number, repeated squaring)
- Any time you need **$O(1)$ space** over a cycle/length problem

- **Skeleton:**

```
# Variant A: Linked-list (cycle detection / middle)
slow, fast = head, head          # both start at head
while fast and fast.next:        # guard: fast.next.next must be safe
    slow = slow.next
    fast = fast.next.next
    if slow is fast:              # cycle detected (omit this for middle)
        return True
return False                      # fast exhausted → no cycle; slow = middle

# Variant B: Abstract sequence (e.g. happy number)
# CRITICAL: offset fast by one step so loop doesn't exit immediately
slow, fast = n, get_next(n)      # ← NOT both = n
while fast != target and slow != fast:
    slow = get_next(slow)
    fast = get_next(get_next(fast))
return fast == target             # True = reached goal, False = cycle hit

# Variant C: Index-based simulation (array middle)
slow, fast = 0, 0
while fast < len(arr) and fast + 1 < len(arr):  # guard 2-step jump
    slow += 1
    fast += 2
return arr[slow:]                 # slow is at the (second) middle index
```

- **Complexity:** $O(N)$ time, $O(1)$ space.

- **Killer Gotchas:**

1. `while fast and fast.next` — always guard both; `fast.next.next` will crash if `fast.next` is `None`.
2. In p202, **fast starts at `get_next(n)` not `n`** — if both started equal, `slow==fast` would trigger on the first check and always return `fast==1` → wrong for unhappy numbers.
3. p876 uses `fast + 1 < len(arr)` (not `fast.next` and `fast.next.next`) when simulating with indices — the guard protects the two-step jump.
4. For even-length lists, `slow` lands on the **second** middle (LeetCode standard). If you need the first middle, stop when `fast.next` is `None` instead.

Problem	Difficulty	Key Trick	Essence
p141 Linked List Cycle	Easy	Both start at <code>head</code> , move then compare	Floyd's: meet → cycle; fast hits <code>None</code> → false
p202 Happy Number	Easy	<code>fast = get_next(n)</code> (offset init); <code>get_next</code> abstracts the step	Chase sequence to 1 or cycle; return <code>fast == 1</code>
p876 Middle of Linked List	Easy	<code>fast+1 < len</code> guards 2-step; <code>slow</code> is result	Fast $2\times$ / slow $1\times$; <code>slow</code> lands on 2nd middle for even-length

hashmap

- **Mindset:** Trade $O(N)$ space for $O(1)$ lookup by mapping a discriminant key (value / count / index) to its data — eliminating rescans and enabling $O(1)$ design operations.
 - **When to use:** count/group frequencies, pair/two-sum lookup, $O(1)$ insert/remove/contains/getRandom, dedupe, “how many share property X”, encode decode bijection, geometry invariants (XOR corner sets).
 - **Complexity:** $O(N)$ time, $O(N)$ space (typical); p447 is $O(N^2)$ due to N pivot passes.
-

Skeleton A — $O(1)$ Full Set (p380)

```
class RandomizedSet:
    def __init__(self):
        self.vals: list[int] = []          # for  $O(1)$  random via random.choice
        self.idx_map: dict[int, int] = {}  # val → index in vals

    def insert(self, val) -> bool:
        if val in self.idx_map: return False
        self.idx_map[val] = len(self.vals)
        self.vals.append(val)
        return True

    def remove(self, val) -> bool:
        if val not in self.idx_map: return False
        idx = self.idx_map[val]
        last = self.vals[-1]
        # Step 1: overwrite slot with last element
        self.vals[idx] = last
        self.idx_map[last] = idx  # fix last's index (no-op if val == last)
        # Step 2: shrink (MUST come after fixing last's index)
        self.vals.pop()
        del self.idx_map[val]
        return True

    def getRandom(self): return random.choice(self.vals)
```

Skeleton B — Reservoir Sampling (p382, p398)

```
# p382: pick 1 uniform random from a stream of unknown length
res, i = head.val, 2
while cur:
    if random.randint(1, i) == 1:  # prob 1/i → keeps each element equally likely
        res = cur.val
    cur, i = cur.next, i + 1

# p398 (alternative - precompute when array fits in memory):
from collections import defaultdict
self.idx = defaultdict(list)
for i, v in enumerate(nums): self.idx[v].append(i)
```

```
def pick(target): return random.choice(self.idx[target])
```

Skeleton C — XOR-Set Geometry Invariant (p391)

```
# Perfect rectangle: two simultaneous conditions must BOTH hold
corners, total_area = set(), 0
for x1, y1, x2, y2 in rectangles:
    total_area += (x2-x1)*(y2-y1)
    for c in [(x1,y1),(x1,y2),(x2,y1),(x2,y2)]:
        corners ^= {c} # toggle: interior corners cancel out
# Valid iff: only 4 outer corners remain AND areas sum to bounding box
assert len(corners) == 4 and corners == {"outer", "4", "corners"} and total_area == W*H
```

Skeleton D — TinyURL Bijection (p535)

```
# Two-way map with base-62 counter (NOT Python hash())
self.url_to_code, self.code_to_url, self.counter = {}, {}, 0
def encode(url):
    if url not in url_to_code:
        code = to_base62(counter); counter += 1
        url_to_code[url] = code; code_to_url[code] = url
    return f"http://tinyurl.com/{url_to_code[url]}"
def decode(short): return code_to_url[short.split("/")[-1]]
```

Killer Gotchas

1. **p380 swap-with-last order:** Fix `idx_map[last] = idx` BEFORE `del idx_map[val]` and `vals.pop()`. Reversing this order corrupts the map when `val == last` (last element being removed) or leaves a dangling index.
2. **p380 self-swap edge case:** When removing the last element in the array, `last == val`, so `idx_map[last] = idx` is a no-op — the code still works correctly because `del idx_map[val]` fires after.
3. **p382/p398 algorithm choice:** Reservoir sampling = $O(N)$ per query, $O(1)$ space. Precompute dict = $O(N)$ build, $O(1)$ query. Use reservoir only when array doesn't fit in memory or the constraint says so.
4. **p391 both conditions required:** Only checking area is insufficient (overlapping sub-rects can still sum correctly). Only checking 4 corners is insufficient (a cross-shaped void can leave 4 corners). You need BOTH invariants.
5. **p447 actual algorithm:** Sort distances per pivot, then count runs with two pointers — NOT a Counter. Formula `count*(count-1)` for ordered pairs still applies.

Problem Table

Problem	Difficulty	Essence	Key Trick
p380 InsertDeleteGetRandom O(1)	Medium	All ops O(1) on a set	list + val→idx; remove = swap-with-last then pop+del
p382 Linked List Random Node	Medium	Uniform random from unknown-length stream	Reservoir: keep node i with prob 1/i via <code>randint(1,i)==1</code>
p391 Perfect Rectangle	Hard	Do sub-rects tile exactly?	XOR-set corners → only 4 outer survive; AND area sum check
p398 Random Pick Index	Medium	Pick uniform random index of target	Precompute <code>defaultdict(list)</code> of indices; <code>random.choice</code>
p447 Number of Boomerangs	Medium	Ordered equidistant triples per pivot	Sort dist list per pivot, run-length count → <code>c*(c-1)</code>
p500 Keyboard Row	Easy	Words typeable on one keyboard row	<code>set(word.lower())</code> row using 3 pre-built row sets
p535 Encode/Decode TinyURL	Medium	Bijjective URL shortener	Two dicts (url code) + base-62 auto-increment counter
p575 Distribute Candies	Easy	Max unique candy types $n/2$	<code>min(len(set(candyType)), n//2)</code> — unique count vs quota

merge_intervals

- **Mindset:** Sort by start once → a single left→right sweep makes every overlap decision local and trivial. Peak concurrency problems split into two sorted arrays and use a two-pointer event simulation instead.
- **When to use:** Any `[start, end]` interval problem — merge overlaps, insert a new interval, count max concurrent meetings, compute covered time.
- **Skeleton:**

Pattern A — Merge / Extend (p056)

```

intervals.sort(key=lambda x: x[0])
merged = [intervals[0][:]]
for start, end in intervals[1:]:
    last = merged[-1]
    if start <= last[1]:           # overlap (touching counts: [1,4],[4,5] → [1,5])
        last[1] = max(last[1], end) # nested interval: must take max, not just end
    else:
        merged.append([start, end])

```

Pattern B — 3-Phase Insert (p057)

Input is already sorted; scan in one pass, no re-sort needed.


```
result, i, n = [], 0, len(intervals)
# Phase 1: intervals strictly before new (end < new.start)
while i < n and intervals[i][1] < new_interval[0]:
    result.append(intervals[i]); i += 1
# Phase 2: merge all overlapping with new (start <= new.end)
while i < n and intervals[i][0] <= new_interval[1]:
    new_interval[0] = min(new_interval[0], intervals[i][0])
    new_interval[1] = max(new_interval[1], intervals[i][1])
    i += 1
result.append(new_interval)
# Phase 3: append the rest untouched
while i < n:
    result.append(intervals[i]); i += 1
```

Pattern C — Peak Concurrency (p253)

Split starts & ends into two separate sorted arrays; simulate events.

```
starts = sorted(i[0] for i in intervals)
ends = sorted(i[1] for i in intervals)
rooms, end_ptr = 0, 0
for start in starts:
    if start >= ends[end_ptr]: # a meeting ended → recycle its room
        end_ptr += 1          # (touching is OK: end=30, start=30 → freed)
    else:
        rooms += 1            # no free room → open a new one
return rooms
# rooms = number of starts that could NOT recycle a freed room
```

Pattern D — Capped Interval Sum (p495)

Input already sorted; each attack contributes `min(duration, gap_to_next)`.

```
total = 0
for i in range(len(time_series) - 1):
    total += min(duration, time_series[i+1] - time_series[i])
return total + duration # last attack always gets full duration
```

- **Complexity:** $O(N \log N)$ time (sort dominates), $O(N)$ space.
 - p057: $O(N)$ time (input already sorted), $O(N)$ space.
 - p495: $O(N)$ time (already sorted), $O(1)$ space.
- **Killer gotchas:**
 1. **Nested intervals (p056/p057):** Always use `max(last.end, end)` — a later interval can be fully contained (`[1,10], [2,3]` → `end=3 < last.end=10`, must keep 10).
 2. **Touching = overlapping (p056):** `[1,4], [4,5]` merges to `[1,5]` because `start <= last.end` uses `<=`, not `<`.
 3. **Touching = NOT overlapping for rooms (p253):** Meeting ending at 30 and next starting at 30 → `start >= ends[end_ptr]` fires → room recycled, no extra room needed.
 4. **p057 boundary conditions:** Phase 1 ends condition is STRICT (`< new.start`); Phase 2 overlap condition is INCLUSIVE (`<= new.end`). Swapping them breaks edge cases.
 5. **p495 last attack:** The loop runs only `n-1` times. Always add `duration` at the end — the final attack is never cut short by a subsequent one.

6. **p253 room count is NOT end_ptr**: `rooms` counts starts that got no free room. At the end, `rooms` = minimum rooms needed (not `end_ptr`).

Problem	Difficulty	Essence	Key trick
p056 Merge Intervals	Medium	Sort by start; extend <code>last.end</code> on overlap	<code>max(last.end, end)</code> handles nesting; <code><=</code> catches touching
p057 Insert Interval	Medium	3-phase linear scan (before / overlap-merge / after)	Phase boundary: strict <code><</code> then inclusive <code><=</code> ; <code>O(N)</code> , no sort
p253 Meeting Rooms II	Medium	Split into sorted <code>starts+ends</code> ; count starts without a free room	<code>start >= ends[ptr]</code> (, not <code>></code>) recycles room; touching = free
p495 Teemo Attacking	Easy	Sum <code>min(duration, gap)</code> for each consecutive pair	Always <code>+duration</code> at end for last attack's full window

sliding_window

- **Mindset**: Maintain a `[left, right]` window. Expand `right` every step; contract `left` only when the window violates the constraint. Each element enters and leaves at most once $\rightarrow O(N)$.
- **When to use**: “Longest/shortest subarray/substring” under a constraint; “count subarrays/substrings satisfying X”; “anagram / permutation in string”; “at most K distinct / replacements”. Key signal: answers involve *contiguous* ranges.

• Skeleton:

```
# --- VARIABLE window (p003, p424, p395) ---
left = 0
state = {}          # freq map / whatever tracks validity
best = 0
for right, ch in enumerate(s):
    # 1. Expand: add s[right] to state
    state[ch] = state.get(ch, 0) + 1

    # 2. Shrink: while window is INVALID, remove s[left]
    while invalid(state):          # e.g. len(state) > k
        state[s[left]] -= 1
        if state[s[left]] == 0:
            del state[s[left]]
        left += 1

    # 3. Update answer
    best = max(best, right - left + 1)
return best

# --- FIXED window (p438, p567) ---
w = len(p)
target = Counter(p)          # or list[26]
```

```

window = Counter(s[:w])
result = [0] if window == target else []
for i in range(w, len(s)):
    window[s[i]] += 1
    window[s[i - w]] -= 1
    if window[s[i - w]] == 0:
        del window[s[i - w]]    # keep Counter clean for == comparison
    if window == target:
        result.append(i - w + 1)

```

- **Complexity:** $O(N)$ time, $O(\text{alphabet})$ space (26 for lowercase, 128 for ASCII).
- **Killer gotchas:**
 - **p003** — jump `left` only if `char_index[ch] >= left`; stale indices (from before the current window) must be ignored, not acted on.
 - **p424** — `max_freq` is **never decremented** when shrinking. It tracks the global historical max, not the current window max. This is intentional: the window only grows when a new `max_freq` is achievable, otherwise it slides (`left+1`, `right+1`). The window size is monotonically non-decreasing.
 - **p438** — delete zero-count keys before `Counter ==` comparison. Python `Counter` treats `{'a': 0} != {}`, so stale zero entries break equality.
 - **p567** — use `list[26]` instead of `Counter` to sidestep zero-key deletion entirely: `list` comparison ignores nothing, and $O(26)$ equality is truly $O(1)$.
 - **p395** — not solvable with a single sliding window pass. Trick: iterate `t` from 1 to 26 (number of distinct chars allowed). For each `t`, use a variable window constrained to exactly `t` unique chars where all have `freq == k`. Total: $26 \times O(N)$.

Problem	Pattern	Essence / Key Trick
p003 Longest Substring No Repeat	Variable	<code>char→last_index</code> ; jump <code>left = last_index+1</code> only if index <code>left</code> (stale guard)
p395 Longest Substring K Repeating	Variable ($\times 26$)	Loop <code>t</code> in <code>1..26</code> ; window with exactly <code>t</code> unique chars, all <code>freq k</code> ; <code>unique==t</code> and <code>at_least_k==t</code>
p424 Longest Repeating Char Replacement	Variable	<code>freq + max_freq</code> ; valid iff <code>window_len - max_freq == k</code> ; <code>max_freq</code> never decreases
p438 Find All Anagrams	Fixed	Rolling Counter vs <code>p_count</code> ; delete zero-count keys so <code>==</code> works correctly
p567 Permutation in String	Fixed	<code>list[26]</code> arrays; roll by <code>+1/-1</code> ; <code>list</code> equality avoids zero-key issue entirely

string

- **Mindset:** Strings are char arrays — normalize (`strip/upper/lower`) then rebuild, or scan with a small state machine. Reach for built-ins (`isupper`, `islower`, `split`) before hand-rolling loops.

Many “clever” string problems reduce to a 1-liner once the right observation is made (p521).

- **When to use:** reformatting/parsing, char-class validation, case rules, counting segments, fixed-size block grouping, license-key / URL transforms, LCS-style puzzles with a mathematical shortcut.
- **Skeleton:**

```
# --- Pattern A: normalize → chunk rebuild (p482) ---
cleaned = s.replace("-", "").upper()          # strip noise, canonicalize case
first_len = len(cleaned) % k                  # FIRST group is shorter (not last!)
groups = []
if first_len > 0:
    groups.append(cleaned[:first_len])
for i in range(first_len, len(cleaned), k):
    groups.append(cleaned[i:i+k])
return "-".join(groups)

# --- Pattern B: in_segment flag (p434) ---
count, in_seg = 0, False
for ch in s:
    if ch != " " and not in_seg:
        count += 1
        in_seg = True
    elif ch == " ":
        in_seg = False
return count
# Alternative one-liner: len(s.split())

# --- Pattern C: built-in char-class predicates (p520) ---
return word.isupper() or word.islower() or word[1:].islower()
# word[1:].islower() handles first-cap + rest-lower AND single-char words

# --- Pattern D: mathematical shortcut (p521) ---
return -1 if a == b else max(len(a), len(b))
# Key: equal STRINGS → -1 (not equal lengths!)
```

- **Complexity:** $O(N)$ time, $O(N)$ space for rebuilt string; $O(1)$ for pure scan/predicate.
- **Killer gotchas:**
 - **p482** — first group is shorter, **not** the last: `first_len = len(cleaned) % k`. Slice `[:first_len]` from the front, then iterate the rest.
 - **p521** — condition is `a == b` (string equality), **NOT** `len(a) == len(b)`. “aaa” vs “bbb”: same length but different → answer is 3, not -1. The insight: if `a == b`, the longer string cannot be a subsequence of the other, so return `max(len)`.
 - **p520** — use `word[1:].islower()` (not `word[0].isupper()` and `word[1:].islower()`): the `[1:]` slice covers single-char words cleanly too.
 - **p434** — `len(s.split())` is the built-in shortcut, but the `in_segment` flag version is the expected “explain your logic” answer in interviews.

Problem	Complexity	Key trick
p434 Number of Segments	$O(N)$ / $O(1)$	<code>in_segment</code> flag: count space→non-space transitions; or just <code>len(s.split())</code>
p482 License Key Formatting	$O(N)$ / $O(N)$	Strip+upper first; first group = <code>len%k</code> (front slice), then k-chunks joined by -
p520 Detect Capital	$O(N)$ / $O(1)$	1-liner: <code>isupper</code> or <code>islower</code> or <code>word[1:].islower()</code> — <code>[1:]</code> covers single chars too
p521 Longest Uncommon Subseq I	$O(N)$ / $O(1)$	<code>a == b</code> (string equality!) → -1; else <code>max(len(a), len(b))</code> — NOT <code>len(a)==len(b)</code>

two_pointers

- **Mindset:** Two indices moving monotonically — once sorted, one comparison tells you which pointer to advance, collapsing $O(N^2)$ into $O(N)$. Three distinct modes: **(A) Inward** (`left=0`, `right=n-1`): pair/triplet sums, container water. **(B) Same-direction** (`slow=0`, `fast=1`): remove dups in-place, k-diff pairs. **(C) Merge two sorted sequences** (i on A, j on B): is-subsequence, heaters.
- **When to use:** Sorted array + pair/triplet sum to target; is-subsequence check; k-diff pairs; container-with-most-water; merge-advance on two sorted arrays.
- **Skeleton:**

```
# (A) Inward - pair sum
left, right = 0, len(nums) - 1
while left < right:
    s = nums[left] + nums[right]
    if s == target: return [left+1, right+1] # 1-indexed for p167!
    elif s < target: left += 1
    else: right -= 1

# (A) 3Sum - fix outer, inner inward, triple dedup
nums.sort()
for i in range(n - 2):
    if i > 0 and nums[i] == nums[i-1]: continue # dedup i
    left, right = i+1, n-1
    while left < right:
        total = nums[i] + nums[left] + nums[right]
        if total < 0: left += 1
        elif total > 0: right -= 1
        else:
            result.append([nums[i], nums[left], nums[right]])
            while left < right and nums[left] == nums[left+1]: left += 1
            while left < right and nums[right] == nums[right-1]: right -= 1
            left += 1; right -= 1 # must move both after recording

# (B) Same-direction - k-diff pairs on sorted array
nums.sort()
```

```

left, right = 0, 1
while right < len(nums):
    diff = nums[right] - nums[left]
    if diff < k: right += 1
    elif diff > k: left += 1
    else:
        count += 1
        lv, rv = nums[left], nums[right]
        while left < len(nums) and nums[left] == lv: left += 1
        while right < len(nums) and nums[right] == rv: right += 1
    if left == right: right += 1 # never let pointers collide

# (C) is_subseq helper - reused in p392, p522, p524
def is_subseq(s: str, t: str) -> bool:
    i = 0
    for ch in t:
        if i < len(s) and s[i] == ch: i += 1
    return i == len(s)

# (C) Merge two sorted - heaters
houses.sort(); heaters.sort()
i = j = 0
while i < len(houses):
    # advance heater j while next heater is closer
    while j+1 < len(heaters) and abs(houses[i]-heaters[j]) >= abs(houses[i]-heaters[j+1]):
        j += 1
    result = max(result, abs(houses[i] - heaters[j]))
    i += 1

```

- **Complexity:** $O(N)$ pair-sum | $O(N^2)$ 3Sum | $O(N \log N)$ k-diff (sort dominates) | $O(N \cdot M)$ is-subseq per pair | $O(1)$ extra space for all pointer variants.

- **Killer gotchas:**

1. **Sort first** — two-pointer logic is invalid on unsorted input (p015, p532, p475).
2. **3Sum triple dedup:** skip `nums[i]==nums[i-1]` for `i`; then skip left/right dups *before* advancing both — a match requires moving both pointers.
3. **p167 is 1-indexed** — return `[left+1, right+1]`, not `[left, right]`.
4. **p532 k=0 guard:** when `k<0` return 0 immediately; when `k=0` pairs are `(x,x)` requiring count 2, handled naturally since `left right` is enforced with `if left==right: right+=1`.
5. **p475 is pure two-pointer, not binary search** — advance heater `j` while next heater is closer or equidistant; never reset `j` (both arrays are co-advancing).
6. **p522 sort-then-check:** sort strings by length descending so longest candidate is checked first; a string is “uncommon” only if it is NOT a subsequence of ANY other string (including duplicates — a duplicate always disqualifies it).
7. **p524 tie-break:** no pre-sorting needed; iterate dictionary, update best if `len(word) > len(best)` OR (same length AND `word < best`) lexicographically.

Problem	Difficulty	Essence	Key Trick
p011 Container With Most Water	Medium	Maximize <code>min(h[l],h[r])*(r-l)</code>	Move the shorter side inward; taller side can't improve area

Problem	Difficulty	Essence	Key Trick
p015 3Sum	Medium	All unique triplets summing to 0	Sort + fix i + inward two-pointer; 3-level dedup
p167 Two Sum II	Medium	Pair sum in sorted array	Inward pointers; return 1-indexed result
p392 Is Subsequence	Easy	Is s embeddable in t by deletions?	is_subseq helper: advance i only on char match
p475 Heaters	Medium	Min radius to cover all houses	Sort both; co-advance heater j while it gets closer; never reset j
p522 Longest Uncommon Subseq II	Medium	Longest string not a subseq of any other	Sort by len desc; use is_subseq ; duplicates always disqualify
p524 Longest Word via Deleting	Medium	Longest dict word that's subseq of s	is_subseq per word; tie-break: longer > lexicographically smaller
p532 K-Diff Pairs	Medium	Count unique pairs with $ diff = k$	Sort + same-dir two-pointer; skip dups after match; guard left==right

binary_search

- **Mindset:** Define a monotone predicate over the search space and halve it each step. One comparison reveals which half the answer is in; shrink toward it until convergence.
- **When to use:** Sorted/rotated array, first/last occurrence, leftmost satisfying a predicate (**isBadVersion**), $O(\log n)$ required, LIS insertion point, “binary search the answer” (search over a value space, not an index space).

Skeleton — Three Templates

```
# Template A - Exact match (return index or -1)
left, right = 0, len(nums) - 1
while left <= right:
    mid = left + (right - left) // 2 # overflow-safe (critical in C/C++/Java)
    if nums[mid] == target:
        return mid
    elif nums[mid] < target:
        left = mid + 1
    else:
        right = mid - 1
return -1
```

```
# Template B - Find leftmost position where predicate turns True
# (converges to the boundary; NEVER use right=mid-1)
```

```

left, right = 0, n          # or (1, n) depending on domain
while left < right:         # NOT <=; loop exits when left==right
    mid = left + (right - left) // 2
    if predicate(mid):
        right = mid        # keep mid; answer might BE mid
    else:
        left = mid + 1
return left                 # left == right == answer

# Template C - Outer loop over candidate lengths/digits + inner exact BS
# Used when you enumerate a parameter m and binary-search a second param k
for m in range(max_m, 1, -1):
    lo, hi = 2, int(num ** (1.0 / (m - 1))) + 2
    while lo <= hi:
        mid = (lo + hi) // 2
        s = geometric_sum(mid, m)
        if s == num: return str(mid)
        elif s < num: lo = mid + 1
        else: hi = mid - 1

```

- **Rotated array (p153):** compare mid vs right (NOT vs left).
 $\text{nums}[\text{mid}] > \text{nums}[\text{right}] \rightarrow \text{min is in right half} \rightarrow \text{left} = \text{mid} + 1$
 $\text{nums}[\text{mid}] \leq \text{nums}[\text{right}] \rightarrow \text{min is in left half (incl. mid)} \rightarrow \text{right} = \text{mid}$
- **Prefix-weight sampling (p497):** build cumulative integer-point counts.
 pick: pick random t in $[1, \text{total}]$, then Template B to find first prefix $\geq t$.
- **Complexity:** $O(\log n)$ time, $O(1)$ space ($O(n \log n)$ for p354 due to sort).
- **p352 addNum:** $O(k)$ amortized per call where k = intervals merged; getIntervals $O(1)$.

Killer Gotchas

1. **Template mismatch = infinite loop.** while $\text{left} < \text{right} + \text{right} = \text{mid}$ is safe. while $\text{left} \leq \text{right} + \text{right} = \text{mid}$ (not $\text{mid}-1$) loops forever when $\text{left} == \text{right}$.
2. $(\text{left} + \text{right}) // 2$ **overflows** in C/C++/Java (32-bit int). Always use $\text{left} + (\text{right} - \text{left}) // 2$. Python is immune (arbitrary precision).
3. **p153 — compare mid vs right, not left.** If you compare $\text{nums}[\text{mid}] < \text{nums}[\text{left}]$ you get wrong answers for the non-rotated case or single-element arrays.
4. **p354 sort key must be (w asc, h desc) for same-width envelopes.** If two envelopes have equal width, the taller one (sorted descending) prevents it from being counted twice in the LIS on heights. (w asc, h asc) gives wrong answers.
5. **p497 counts integer POINTS, not area.** Width contribution is $(x_2 - x_1 + 1)$, not $(x_2 - x_1)$. Off-by-one here breaks the uniform distribution.
6. **p483 fallback is $n-1$, not 2.** Any integer $n \geq 3$ written in base $n-1$ is always "11", so the answer is at most $n-1$.

Problem	Difficulty	Essence	Key trick
p704 Binary Search	Easy	Classic exact-match	Template A; return mid on hit, -1 after loop
p153 Find Min Rotated	Medium	Boundary of rotation	$\text{nums}[\text{mid}] > \text{nums}[\text{right}] \rightarrow \text{left} = \text{mid} + 1$ else $\text{right} = \text{mid}$; return $\text{nums}[\text{left}]$

Problem	Difficulty	Essence	Key trick
p278 First Bad Version	Easy	Leftmost True predicate	Template B; <code>is_bad(mid)→right=mid;</code> <code>else left=mid+1</code>
p352 Data Stream Disjoint Intervals	Hard	Sorted interval list + merge on insert	<code>bisect_left</code> on <code>[lo,hi]</code> pairs; merge left if <code>prev[1]>=lo-1</code> ; merge right while <code>cur[0]<=hi+1</code>
p354 Russian Doll Envelopes	Hard	2-D LIS → 1-D LIS	Sort (<code>w↑</code> , <code>h↓</code>); <code>bisect_left</code> on <code>heights</code> ; replace or append (patience sort)
p483 Smallest Good Base	Hard	Geometric sum = n ?	Outer loop <code>m</code> from <code>bit_length(n)</code> down to 2; inner BS <code>k</code> in <code>[2, n^(1/(m-1))+2]</code> ; fallback = <code>n-1</code>
p497 Random Point in Rectangles	Medium	Weighted random rect	Prefix integer-point counts; <code>randint(1,total)</code> ; Template B to find rect; <code>randint</code> within rect

Skeleton — Three Templates

bit_manipulation

- **Mindset:** Treat numbers as bit vectors; exploit XOR/AND/shift identities ($a \oplus a = 0$, $a \oplus 0 = a$, $a \& (a-1)$ clears lowest set bit) instead of arithmetic. Pick the right skeleton: XOR-fold for “find the odd one out”, Kernighan loop for “count set bits”, DP-on-bits for “all counts 0..n”.
- **When to use:** “every element appears twice except one”, “missing number”, parity / counting bits, power-of-two check ($n \& (n-1) == 0$), mask/flip complement, $O(1)$ space required, “without division”.
- **Skeleton:**

```
# 1. XOR fold - cancel duplicate pairs, isolate unique element
res = 0
for x in nums:
    res ^= x          # a^a=0, a^0=a → only odd-count element survives

# 2. Brian Kernighan - count set bits in O(popcount) iterations
count = 0
while n:
    n &= n - 1        # clears the LOWEST set bit each iteration
    count += 1        # loops exactly popcount(n) times, not bit_width times

# 3. DP on bits - build answer array for 0..n in O(n)
ans = [0] * (n + 1)
```

```
for i in range(1, n + 1):
    ans[i] = ans[i >> 1] + (i & 1)    # i>>1 drops LSB; i&1 is that dropped bit

# 4. Full-width complement mask - flip exactly the significant bits
mask = (1 << num.bit_length()) - 1    # e.g. num=5 (101) → mask=7 (111)
complement = num ^ mask                # flips only the meaningful bits
# Only safe when num >= 1 (bit_length(0)==0 → mask=0, complement=0)
```

- **Complexity:** O(n) time, O(1) space (O(n) space for DP array variant). Kernighan loop is O(k) where k = number of set bits bit-width.
- **Killer gotchas:**
 1. **~num in Python is NOT bit-flip** — it returns -(num+1) (unlimited-precision two's complement). Always build an explicit mask with `bit_length()` and XOR (p476).
 2. **n &= n-1 clears the lowest set bit**, not just “subtracts 1”. This is why the loop runs in O(popcount) time, not O(32). Know this distinction cold.
 3. **bit_length() edge case:** (0).bit_length() == 0, so mask = 0 and the complement formula breaks for num=0. Constraints usually exclude 0 — verify first.
 4. **Power-of-two check:** n > 0 and n & (n-1) == 0. The n > 0 guard is essential because 0 & -1 == 0 would incorrectly pass without it.

Problem	Essence	Key trick
p136 Single Number	XOR all — pairs cancel	res=0; for x in nums: res^=x — O(n)/O(1)
p0191 Number of 1 Bits	Count set bits	Brian Kernighan: n &= n-1 loops exactly popcount(n) times
p338 Counting Bits	Bit counts for 0..n	DP: ans[i] = ans[i>>1] + (i&1) — right-shift shares subproblem
p476 Number Complement	Flip significant bits only	mask=(1<<num.bit_length())- 1; return num^mask — avoids Python ~ trap

design

- **Mindset:** Layer multiple O(1) data structures (dicts, sets, `OrderedDict`) so every operation routes to a cheap lookup — trade memory for speed. The hard part is keeping auxiliary structures in sync on every mutation.
- **When to use:** “design a class”, “implement X with O(1) avg”, LRU/LFU, eviction policies, hit-counter streams, any “cache with policy” problem.
- **Skeleton (Simple LFU — as implemented):**

```
from collections import defaultdict

class LFUCache:
    def __init__(self, capacity: int):
        self.cap = capacity
        self.key_to_val = {}          # key → value
```

```
self.key_to_freq = {}          # key → current frequency
self.freq_to_keys = {}        # freq → set of keys at that freq

def _update_freq(self, key: int):
    freq = self.key_to_freq[key]
    self.freq_to_keys[freq].discard(key)
    if not self.freq_to_keys[freq]:          # ← MUST delete empty bucket
        del self.freq_to_keys[freq]         # or min() returns stale freq
    self.key_to_freq[key] = freq + 1
    self.freq_to_keys.setdefault(freq + 1, set()).add(key)

def get(self, key: int) -> int:
    if key not in self.key_to_val:
        return -1
    self._update_freq(key)
    return self.key_to_val[key]

def put(self, key: int, value: int) -> None:
    if self.cap <= 0:                # ← guard: capacity 0 edge case
        return
    if key in self.key_to_val:        # UPDATE existing key
        self.key_to_val[key] = value
        self._update_freq(key)        # no eviction needed
    else:                             # INSERT new key
        if len(self.key_to_val) >= self.cap: # evict first
            min_freq = min(self.freq_to_keys) # O(k) - not truly O(1)
            evict = self.freq_to_keys[min_freq].pop()
            del self.key_to_val[evict]
            del self.key_to_freq[evict]
        self.key_to_val[key] = value
        self.key_to_freq[key] = 1
        self.freq_to_keys.setdefault(1, set()).add(key)
```

- Skeleton (True O(1) LFU — interview gold standard):

```
from collections import defaultdict, OrderedDict

class LFUCache:
    def __init__(self, capacity: int):
        self.cap = capacity
        self.min_freq = 0          # ← tracked variable, reset to 1 on new insert
        self.key_to_val = {}
        self.key_to_freq = {}
        self.freq_to_keys = defaultdict(OrderedDict) # freq → {key: None} in LRU order

    def _update_freq(self, key: int):
        freq = self.key_to_freq[key]
        del self.freq_to_keys[freq][key]
        if not self.freq_to_keys[freq] and freq == self.min_freq:
            self.min_freq += 1      # ← safe: only one freq can be min
        self.key_to_freq[key] = freq + 1
        self.freq_to_keys[freq + 1][key] = None
```

```
def get(self, key: int) -> int:
    if key not in self.key_to_val:
        return -1
    self._update_freq(key)
    return self.key_to_val[key]

def put(self, key: int, value: int) -> None:
    if self.cap <= 0:
        return
    if key in self.key_to_val:
        self.key_to_val[key] = value
        self._update_freq(key)
    else:
        if len(self.key_to_val) >= self.cap:
            evict, _ = self.freq_to_keys[self.min_freq].popitem(last=False) # LRU
            del self.key_to_val[evict]
            del self.key_to_freq[evict]
        self.key_to_val[key] = value
        self.key_to_freq[key] = 1
        self.freq_to_keys[1][key] = None
        self.min_freq = 1 # ← new key always starts at freq=1
```

- **Complexity:**

- Simple variant: $O(k)$ eviction (k = distinct freqs), $O(1)$ get/update, $O(\text{capacity})$ space
- True $O(1)$ variant: $O(1)$ all ops amortized, $O(\text{capacity})$ space

- **Killer gotchas:**

1. **Must delete empty freq buckets** — otherwise `min(freq_to_keys)` or `min_freq` tracking returns a stale frequency and evicts the wrong key.
2. **`min_freq` always resets to 1 after a new put** — every newly inserted key starts at `freq=1`, which is always any existing freq. Skip this reset and you evict the wrong bucket.
3. **Update Insert** — if key already exists in `put()`, just update value and bump freq; do NOT evict. Failing this double-counts capacity.
4. **`cap <= 0` guard** — LeetCode tests with `capacity=0`; skip all ops or you'll evict into an empty dict.
5. **LRU tie-break within same freq** — plain `set.pop()` is arbitrary, not LRU. Only `OrderedDict` with `popitem(last=False)` gives correct LRU order within a frequency bucket.

Problem	Difficulty	Essence	Key trick
p460 LFU Cache	Hard	3-dict LFU: key→val, key→freq, freq→bucket	Delete empty buckets; track <code>min_freq</code> ; <code>OrderedDict</code> per bucket for true $O(1)$ LRU tie-break

dfs

- **Mindset:** Explore as deep as possible before backtracking. Mark nodes visited **before** recursing so no node is processed twice. Return values bubble up through the call stack — use this to aggregate (sum, count, bool) in a single pass.
 - **When to use:** Grid connectivity (islands/regions/flood-fill), tree traversals (pre/in/post-order), BST search & restructure, graph reachability over index graphs, parsing nested structures iteratively via an explicit stack.
-

- **Skeleton:**

```
# Grid DFS (sink/flood pattern)
def num_islands(grid):
    rows, cols = len(grid), len(grid[0])
    def dfs(r, c):
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != "1":
            return
        grid[r][c] = "0"          # mark BEFORE recursing!
        dfs(r+1,c); dfs(r-1,c); dfs(r,c+1); dfs(r,c-1)
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1": count += 1; dfs(r, c)
    return count

# Grid DFS returning aggregate
def dfs_area(r, c):
    if out_of_bounds or grid[r][c] != 1: return 0
    grid[r][c] = 0
    return 1 + dfs(r+1,c) + dfs(r-1,c) + dfs(r,c+1) + dfs(r,c-1)

# Tree DFS (post-order aggregate)
def subtree_sum(node):
    if not node: return 0
    s = node.val + subtree_sum(node.left) + subtree_sum(node.right)
    freq[s] = freq.get(s, 0) + 1
    return s

# Tree DFS (structural match)
def is_same(a, b):
    if not a and not b: return True
    if not a or not b: return False
    return a.val == b.val and is_same(a.left, b.left) and is_same(a.right, b.right)

def is_subtree(root, sub):
    if not sub: return True
    if not root: return False
    return is_same(root, sub) or is_subtree(root.left, sub) or is_subtree(root.right, sub)

# BST DFS (search + modify)
def delete_bst(root, key):
```

```
if not root: return None
if key < root.val:
    root.left = delete_bst(root.left, key)
elif key > root.val:
    root.right = delete_bst(root.right, key)
else:
    # found
    if not root.left: return root.right # 0 or 1 child
    if not root.right: return root.left
    successor = root.right # inorder successor
    while successor.left: successor = successor.left
    root.val = successor.val # overwrite with successor
    root.right = delete_bst(root.right, successor.val)
return root

# BST DFS (reverse in-order accumulation)
def convert_to_greater(root):
    total = 0
    def reverse_inorder(node):
        nonlocal total
        if not node: return
        reverse_inorder(node.right) # visit larger values first
        total += node.val; node.val = total
        reverse_inorder(node.left)
    reverse_inorder(root)

# Index-graph DFS (follow chain)
def jump_game(arr, start):
    visited = set()
    def dfs(i):
        if i < 0 or i >= len(arr) or i in visited: return False
        if arr[i] == 0: return True
        visited.add(i)
        return dfs(i + arr[i]) or dfs(i - arr[i])
    return dfs(start)

# Permutation cycle (iterative, no recursion)
def array_nesting(nums):
    # p565 - NOT recursive DFS
    visited = [False] * len(nums)
    max_len = 0
    for i in range(len(nums)):
        if not visited[i]:
            count, j = 0, i
            while not visited[j]:
                visited[j] = True; j = nums[j]; count += 1
            max_len = max(max_len, count)
    return max_len

# Stack-based parser (iterative, DFS-flavored)
def deserialize(s):
    # p385 - iterative stack, not recursive
    if not s.startswith("["): return NestedInteger(int(s))
    stack = []
    i = 0
```

```
while i < len(s):
    if s[i] == "[": stack.append(NestedInteger()); i += 1
    elif s[i] == "]":
        done = stack.pop()
        if stack: stack[-1].add(done)
        else: return done
        i += 1
    elif s[i] == ",":
        i += 1
    else:
        j = i
        while j < len(s) and (s[j].isdigit() or s[j] == "-"): j += 1
        if stack: stack[-1].add(NestedInteger(int(s[i:j])))
        i = j
return stack[0]

# Stack-as-depth (cumulative length)
def longest_file_path(input_str): # p388
    stack = [0] # stack[depth] = cumulative path length up to that depth
    max_len = 0
    for part in input_str.split("\n"):
        depth = part.count("\t")
        name = part.lstrip("\t")
        while len(stack) > depth + 1: stack.pop()
        if "." in name: max_len = max(max_len, stack[-1] + len(name))
        else: stack.append(stack[-1] + len(name) + 1) # +1 for "/"
    return max_len
```

- **Complexity:**

- Grid DFS: $O(\text{rows} \times \text{cols})$ time, $O(\text{rows} \times \text{cols})$ space (call stack worst case)
- Tree DFS: $O(N)$ time, $O(H)$ space (H = height; $O(N)$ worst for skewed tree)
- BST delete: $O(H)$ time and space
- Index/cycle DFS: $O(N)$ time, $O(N)$ space (visited set/array)
- Parser/file-path: $O(|s|)$ time, $O(\text{depth})$ space

- **Killer gotchas:**

1. **Mark visited BEFORE recursing** — `grid[r][c] = "0"` must come *before* the 4 recursive calls, or you get infinite loops revisiting the same cell.
2. **Python recursion limit** — grids up to $300 \times 300 = 90,000$ cells; default limit is 1000. Either `sys.setrecursionlimit(200000)` or go iterative with an explicit stack.
3. **BST delete with 2 children** — don't just find the inorder successor; you must also **recursively delete it from the right subtree** (`root.right = delete(root.right, successor.val)`). Forgetting this leaves a duplicate.
4. **p565 is NOT recursive DFS** — it's an iterative cycle-following loop. Don't try to recurse on large inputs (n up to $10^5 \rightarrow$ stack overflow).
5. **p385 is stack-based, NOT recursive descent** — the stack simulates DFS nesting; push on `[`, pop on `]`, parse number token inline.
6. **p388 stack stores cumulative lengths, not names** — `stack[d]` = total char count of path up to depth `d`. Add `+1` for each `/` separator when pushing directories.

Problem	Essence	Key trick
p200 Number of Islands	Count connected '1' components	Sink cells ('1' → '0') in-place during DFS; no visited set needed
p695 Max Area of Island	Largest connected 1 component	<code>dfs</code> returns <code>1 + sum(4 neighbors)</code> ; sink to 0 before recursing
p572 Subtree of Another Tree	Does <code>subRoot</code> appear as an exact subtree?	Two-level DFS: outer <code>is_subtree</code> walks all nodes; inner <code>is_same</code> checks structural equality
p508 Most Freq Subtree Sum	Post-order sum → frequency map	<code>subtree_sum</code> returns int and side-effects <code>freq</code> dict; filter by <code>max(freq.values())</code>
p538 Convert BST → Greater Tree	Each node gets += sum of all larger nodes	Reverse in-order (R → N → L) with <code>nonlocal total</code> accumulator; modifies in-place
p450 Delete Node in BST	BST delete handling 3 cases	2-child case: copy inorder successor value up, then recursively delete successor from right subtree
p1306 Jump Game III	Can index with value 0 be reached via $\pm$ jumps?	DFS on index graph; <code>visited.add(i)</code> before branching to avoid cycles
p565 Array Nesting	Longest cycle in permutation array	Iterative cycle-follow with boolean visited array; mark before advancing <code>j = nums[j]</code>
p385 Mini Parser	Deserialize "[123, [456, [789]]]" string	Iterative stack: push on [, pop+attach on], parse digit runs inline
p388 Longest Abs File Path	Longest path to a file in indented filesystem string	Stack of cumulative path lengths indexed by depth; +1 per /; file = contains .

divide_and_conquer

- **Mindset:** Split input into independent halves, solve each recursively, then combine. The combine step is where the real work (merge, count, build) happens. Think: `solve(whole) = combine(solve(left), solve(right))`.
- **When to use:** Sorting, “merge k ...”, recurrence $T(n) = 2T(n/2) + O(n)$, counting reverse pairs / inversions, 2-D quad-tree partition. Also when a problem is tagged D&C but has a cleaner greedy/iterative solution (e.g., p169 Boyer-Moore).
- **Complexity:** $O(n \log n)$ time, $O(n)$ merge buffer + $O(\log n)$ stack. Exception: p427 `all_same()` scan is $O(\text{size}^2)$ per call → worst-case $O(n^2 \log n)$.

Skeleton A — Merge Sort (p912, p023)


```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr[:]          # base case: already sorted copy
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    # two-pointer merge
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Skeleton B — Count-Then-Merge (p493 Reverse Pairs)

```
def merge_sort_count(arr):
    if len(arr) <= 1:
        return 0
    mid = len(arr) // 2
    left, right = arr[:mid], arr[mid:]
    count = merge_sort_count(left) + merge_sort_count(right)
    # Phase 1: COUNT cross-half pairs BEFORE sorting (values still original)
    j = 0
    for num in left:
        while j < len(right) and num > 2 * right[j]: # ← condition varies
            j += 1
        count += j
    # Phase 2: SORT the subarray (destroys original order - must count first!)
    arr[:] = sorted(left + right) # or use two-pointer merge for O(n)
    return count
```

Skeleton C — 2-D Quad-Tree (p427)

```
def build(x, y, size):
    if all_same(x, y, size):          # O(size²) scan
        return leaf_node(grid[x][y])
    half = size // 2
    tl = build(x, y, half)
    tr = build(x, y + half, half)
    bl = build(x + half, y, half)
    br = build(x + half, y + half, half)
    return internal_node(tl, tr, bl, br)
```

- **Complexity:** $O(n \log n)$ time, $O(n)$ buffer + $O(\log n)$ stack.
- **Killer gotchas:**
 1. **Slicing doubles memory:** `arr[:mid]` creates a copy at every level $\rightarrow O(n \log n)$ total extra space; use index ranges if memory-bound.

2. **p493 two-phase order:** COUNT cross pairs with original values FIRST, THEN sort. Swapping the phases gives wrong answers because sorting destroys the pair relationship.
3. **p493 condition is $> 2 \times \text{right}[j]$** , NOT a plain inversion ($> \text{right}[j]$). The $2 \times$ multiplier makes it a “reverse pair”, not a classic inversion count.
4. **p169 Boyer-Moore prerequisite:** The algorithm is guaranteed correct only when a strict majority ($> n/2$ occurrences) is guaranteed to exist. Without that guarantee, a second validation pass is required.
5. **p427 all_same() cost:** $O(\text{size}^2)$ per call; full-grid uniform = $O(1)$ nodes but worst-case (all mixed) = $O(n^2 \log n)$. Grid size is constrained to $n = 2^x$ ($x \leq 6$) so this is fine in practice, but don’t assume $O(n \log n)$ for large grids.

Problem	Essence	Key trick
p912 Sort an Array	Classic merge sort	Split at mid, recurse, two-pointer merge; base len 1 returns a copy
p023 Merge k Sorted Lists	D&C on the list-of-lists array	Split lists array in half, recurse each half to a flat sorted array, two-pointer merge
p169 Majority Element	Boyer-Moore voting (NOT actual D&C)	count==0 $\rightarrow$ new candidate; works only when majority $> n/2$ is guaranteed
p427 Construct Quad Tree	Recursive 2-D subdivision	all_same(x,y,size) $\rightarrow$ leaf; else split into 4 quadrants; BFS serialize with nulls
p493 Reverse Pairs	Merge-sort counting with $2 \times$ condition	Count left[i] $> 2 \times \text{right}[j]$ cross-pairs (two-pointer) BEFORE sorting the subarray

Skeleton A — Merge Sort (p912, p023)

dynamic_programming

- **Mindset:** Identify state = answer to a sub-instance. Write the recurrence. Choose top-down (memoised recursion) or bottom-up (fill table). Read answer from one cell.
- **When to use:** “max/min”, “number of ways”, “is it possible”, overlapping subproblems
 - optimal substructure; longest/shortest subsequence/path; counting combinations.
- **Complexity:** $O(\text{states} \times \text{transitions})$; space often reducible with rolling arrays.

Skeletons

```
# 1D rolling (Fibonacci)
a, b = 1, 2
for _ in range(3, n + 1):
    a, b = b, a + b      # p070

prev, curr = 0, 0
for num in nums:
```

```
    prev, curr = curr, max(curr, prev + num) # p198

# Unbounded knapsack - MIN (order doesn't matter)
INF = amount + 1 # tighter than float('inf')
dp = [INF] * (amount + 1)
dp[0] = 0
for i in range(1, amount + 1):
    for coin in coins:
        if coin <= i:
            dp[i] = min(dp[i], dp[i - coin] + 1)
return dp[amount] if dp[amount] != INF else -1 # p322

# Unbounded knapsack - COUNT COMBINATIONS (outer=coins!)
dp = [0] * (amount + 1); dp[0] = 1
for coin in coins: # ← outer loop over coins = NO permutation duplicates
    for a in range(coin, amount + 1):
        dp[a] += dp[a - coin] # p518

# Interval DP
n = len(s)
dp = [[0]*n for _ in range(n)]
for i in range(n): dp[i][i] = 1 # base: single char
for length in range(2, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1
        if s[i] == s[j]:
            dp[i][j] = dp[i+1][j-1] + 2
        else:
            dp[i][j] = max(dp[i+1][j], dp[i][j-1]) # p516

# Inverted DP (p887 Super Egg Drop)
# Reframe: dp[m][k] = max floors testable with m moves, k eggs
# Recurrence: dp[m][k] = dp[m-1][k-1] + dp[m-1][k] + 1
# Find smallest m such that dp[m][k] >= n → O(k·log n)
dp = [0] * (k + 1)
moves = 0
while dp[k] < n:
    moves += 1
    for i in range(k, 0, -1):
        dp[i] = dp[i] + dp[i-1] + 1
return moves

# 3D state-machine DP (p552)
# State: dp[i][absences][consecutive_lates]
dp = [[[0]*3 for _ in range(2)] for _ in range(n+1)]
dp[0][0][0] = 1
for i in range(n):
    for a in range(2):
        for cl in range(3):
            v = dp[i][a][cl]
            dp[i+1][a][0] = (dp[i+1][a][0] + v) % MOD # P (present)
            if a < 1:
```

```

    dp[i+1][a+1][0] = (dp[i+1][a+1][0] + v) % MOD # A (absent)
if cl < 2:
    dp[i+1][a][cl+1] = (dp[i+1][a][cl+1] + v) % MOD # L (late)

```

• Killer Gotchas:

1. **Iteration order — combinations vs permutations (p518):** outer loop over **coins**, inner over amounts $\rightarrow$ combinations. Swap them $\rightarrow$ permutations (wrong for p518). p322 can use either order (only counts fewest).
2. **Interval DP must iterate by length (p516):** $dp[i+1][j-1]$ must be filled before $dp[i][j]$. Always outer-loop on **length**, not on i or j .
3. **INF sentinel for min-DP (p322):** use `amount + 1` (not 0, not -1). Return -1 if still INF after fill.
4. **Inverted framing (p887):** the naive $O(kn^2)$ DP (try every floor for the drop) TLEs. Invert: ask “with m moves and k eggs, how many floors can I certify?” — recurrence $dp[m][k] = dp[m-1][k-1] + dp[m-1][k] + 1$. Binary search / increment on m .
5. **Circular rotation formula (p514):** $cost = \min(diff, ring_len - diff) + 1$ — the +1 is the button press and is easy to forget.
6. **Wraparound condition (p467):** consecutive iff $ord(s[i]) - ord(s[i-1]) == 1$ OR -25 ($z \rightarrow a$ wrap). Track $max_len[c] = \max$ run length ending at char c ; $answer = \sum(max_len)$.
7. **p553 is NOT interval DP — it’s pure greedy math:** always place one set of parens: $nums[0] / (nums[1] / (nums[2] / \dots / nums[-1]))$. No DP needed.
8. **p466 is cycle-detection, not classical DP:** simulate $s2$ matching through $s1$ copies, record $s2_index$ at end of each $s1$ copy, detect repeat $\rightarrow$ skip full cycles.

Problem	Pattern	Essence	Key Trick
p070 Climbing Stairs	1D linear	$dp[i] = dp[i-1] + dp[i-2]$	Two rolling vars; this is Fibonacci
p198 House Robber	1D linear	$curr = \max(curr, prev + num)$	Rob-or-skip; no array needed
p322 Coin Change	Unbounded knapsack (min)	$dp[i] = \min(dp[i-coin]+1)$	Init with <code>amount+1</code> ; return -1 if still INF
p518 Coin Change II	Unbounded knapsack (count)	$dp[a] += dp[a-coin]$	Outer loop = coins $\rightarrow$ combinations not permutations
p516 Longest Palindromic Subseq	Interval DP	Match ends +2; else max of shrink one side	Iterate by length; base $dp[i][i]=1$
p403 Frog Jump	Top-down memo	$can_reach(pos, k) \rightarrow$ try $k-1/k/k+1$	Use <code>set(stones)</code> for $O(1)$ lookup; prune <code>step > 0</code>
p887 Super Egg Drop	Inverted DP	$dp[m][k] = dp[m-1][k-1] + dp[m-1][k] + 1$	Reframe: “max floors with m moves”; find $\min m \quad n$
p514 Freedom Trail	1D backward DP	$dp[pos] = \min(rotate_cost + dp[target])$	Circular cost $\min(diff, n-diff)+1$; iterate key backward
p552 Student Attendance II	3D state-machine	$dp[i][absences][consec_late]$	3 late conditions: P/A/L; mod 10 +7

Problem	Pattern	Essence	Key Trick
p576 Out of Boundary Paths	3D grid DP	<code>dp[move][r][c]</code> paths remaining	Count out-of-bounds moves as exits; mod $10^9 + 7$
p467 Unique Wrap Substrings	Char-endpoint DP	<code>max_len[c]</code> = max run length ending at <code>c</code>	Wraparound: <code>diff==1</code> or <code>diff==-25</code> ; <code>answer=sum</code>
p466 Count Repetitions	Cycle detection	Simulate <code>s2</code> match through <code>s1</code> copies; detect <code>s2_index</code> cycle	Skip full cycles; check charset subset first
p553 Optimal Division	Greedy (not DP!)	Always $a/(b/c/\dots/z) = a*c*\dots*z/b$	Math insight: maximise divisor = group all after first under one paren

math

- **Mindset:** Recognize the underlying math structure (information theory, geometry, permutation, palindrome construction) and either apply a closed-form directly or enumerate a small structured candidate set — never simulate exhaustively.
- **When to use:** “minimum number of ...” (information-theoretic lower bound), circle/ geometry sampling, “next permutation”, modular arithmetic, palindrome construction, constraints that hint at a formula ($n \leq 8$, digits ≤ 18).
- **Skeleton:**

```
# p458 - information-theoretic: each pig encodes one "digit" in base (tests+1)
tests = minutesToTest // minutesToDie
pigs = 0
while (tests + 1) ** pigs < buckets:
    pigs += 1
return pigs
# Closed-form equivalent: math.ceil(math.log(buckets) / math.log(tests + 1))
```

```
# p478 - rejection sampling in unit square, then scale
while True:
    x, y = random.uniform(-1, 1), random.uniform(-1, 1)
    if x*x + y*y <= 1: # discard corners; ~21.5% rejection rate
        return [cx + x*r, cy + y*r]
```

```
# p479 - mirror top half → 2n-digit palindrome, then find factor in [√pal, upper]
upper = 10**n - 1
for left in range(upper, lower-1, -1):
    pal = mirror(left) # e.g. 98 → 9889
    for right in range(upper, int(pal**0.5)-1, -1):
        if right*right < pal: break
        if pal % right == 0: return pal % 1337
# Edge case: n==1 → return 9 directly
```

```
# p556 - standard next-permutation on digit list
```

```

digits = list(str(n))
i = len(digits)-2
while i >= 0 and digits[i] >= digits[i+1]: i -= 1
if i < 0: return -1
j = len(digits)-1
while digits[j] <= digits[i]: j -= 1
digits[i], digits[j] = digits[j], digits[i]
digits[i+1:] = reversed(digits[i+1:])
result = int("".join(digits))
return result if result <= 2**31 - 1 else -1

# p564 - generate 5 candidates, pick closest (ties → smaller)
prefix = int(n[: (length+1)//2])
for p in (prefix-1, prefix, prefix+1): # same-length palindromes
    candidates.add(mirror(p, length))
candidates.add(10**(length-1) - 1) # e.g. "999" (one digit shorter)
candidates.add(10**length + 1) # e.g. "1001" (one digit longer)
candidates.discard(num)
return str(min(candidates, key=lambda x: (abs(x-num), x))) # tie → smaller

```

- **Complexity:** | Problem | Time | Space | —|—|—| | p458 | $O(\log \text{ buckets} / \log(\text{tests}+1))$ — tiny loop | $O(1)$ | | p478 | $O(1)$ amortized per point (1.27 tries avg) | $O(1)$ | | p479 | $O(10^n)$ outer $\times O(10^n / \sqrt{\text{pal}})$ inner; n 8, exits early | $O(1)$ | | p556 | $O(d)$ where d = number of digits 10 | $O(d)$ | | p564 | $O(\text{length})$ to build candidates | $O(1)$ |

- **Killer gotchas:**

1. **p458 “+1 state”:** `tests = minutesToTest // minutesToDie` (integer division!). The base is `tests+1` not `tests` — the +1 is the “survived all rounds” state.
2. **p478 naive radius:** `random.uniform(0, r)` for radius clusters points near center (area r^2). Must use rejection sampling in the bounding square, not polar coords with uniform radius.
3. **p479 n=1 edge case:** the mirror loop produces `pal = 1*10+1 = 11` for `left=1`, which has no single-digit factor — hard-code `return 9`.
4. **p556 overflow:** Python ints don’t overflow, but must explicitly check `result <= 2**31 - 1`; the constraint says $n \leq 2^{31}-1$ but the *answer* must also fit.
5. **p564 tie-breaking:** on equal distance, return the **smaller** palindrome. The `key=lambda x: (abs(x-num), x)` tuple sort handles this automatically.
6. **p564 num 10 edge case:** return `str(num - 1)` directly (e.g. “1”→“0”, “9”→“8”) — the generic prefix mirroring breaks down for single-digit inputs.

Problem	Essence	Key trick
p458 Poor Pigs	Min pigs s.t. $(\text{tests}+1)^{\text{pigs}}$ buckets	Information theory: each pig = 1 digit in base <code>tests+1</code> ; loop until capacity buckets
p478 Random Point in Circle	Uniform random point inside circle of radius r	Rejection sample in unit square $[-1,1]^2$; ~78.5% acceptance; then scale by r
p479 Largest Palindrome Product	Largest palindrome = product of two n-digit numbers; return mod 1337	Mirror top-half → candidate palindrome; scan right from upper until <code>right² < pal</code> ; n=1 → 9

Problem	Essence	Key trick
p556 Next Greater Element III	Next permutation of digits; -1 if none or $> 2^{31}-1$	Standard next-permutation: find dip from right, swap with next-larger, reverse suffix
p564 Find Closest Palindrome	Nearest palindrome (not itself); tie $\rightarrow$ smaller	5 candidates: prefix ± 1 mirrored, same prefix mirrored, $10^{(\text{len}-1)-1}$, $10^{\text{len}+1}$

prefix_sum

- **Mindset:** Precompute cumulative aggregates so any contiguous-range query collapses to a difference of two prefix values in $O(1)$. For counting problems, store the prefix value in a hash map and look up `curr - target` to find matching earlier prefixes.
- **When to use:** “subarray sum/product equals k ”, range sum queries, “equal # of 0s and 1s”, $O(n)$ with division banned, weighted random pick, “subarray sum multiple of k ”.
- **Skeleton A — Prefix-freq map (count subarrays summing to k):**

```
# p0560 pattern
freq = defaultdict(int)
freq[0] = 1          # ← MANDATORY base case: empty prefix has sum 0
curr, ans = 0, 0
for x in nums:
    curr += x
    ans += freq[curr - k]  # how many earlier prefixes make this window sum == k
    freq[curr] += 1
```

- **Skeleton B — Prefix-first-index map (earliest index per value):**

```
# p0525 pattern (longest subarray)
seen = {0: -1}        # ← base case: count=0 seen before index 0
count = max_len = 0
for i, x in enumerate(nums):
    count += 1 if x == 1 else -1  # map 0 $\leftrightarrow$ -1 so balance=0 means equal
    if count in seen:
        max_len = max(max_len, i - seen[count])
    else:
        seen[count] = i          # only store FIRST occurrence

# p0523 pattern (existence, length 2, multiples of k)
mod_idx = {0: -1}     # ← base case: remainder 0 seen before index 0
prefix = 0
for i, x in enumerate(nums):
    prefix = (prefix + x) % k
    if prefix in mod_idx:
        if i - mod_idx[prefix] > 1:  # ← MANDATORY: subarray length 2
            return True
        # do NOT update mod_idx[prefix] - keep the FIRST occurrence
    else:
        mod_idx[prefix] = i
return False
```

- **Skeleton C — Two-pass prefix/suffix product (no division):**

```
# p0238 pattern
n = len(nums)
output = [1] * n
left = 1
for i in range(n):
    output[i] = left      # assign BEFORE multiplying
    left *= nums[i]
right = 1
for i in range(n - 1, -1, -1):
    output[i] *= right    # assign BEFORE multiplying (same pattern, right side)
    right *= nums[i]
```

- **Skeleton D — Prefix sum + binary search (weighted random pick):**

```
# p0528 pattern
# __init__: build prefix sums
prefix = []
total = 0
for x in w:
    total += x
    prefix.append(total)
# pickIndex: pick random int in [1, total], binary-search for first prefix >= t
t = random.randint(1, total)    # 1-indexed! not 0-indexed
lo, hi = 0, len(prefix) - 1
while lo < hi:
    mid = (lo + hi) // 2
    if prefix[mid] >= t: hi = mid
    else: lo = mid + 1
return lo
```

- **Complexity:** | Problem | Time | Space | |—|—|—| | p0560, p0525, p0523 | $O(n)$ | $O(n)$ hash | | p0238 | $O(n)$ | $O(1)$ extra (output array doesn't count) | | p0713 | $O(n)$ | $O(1)$ | | p0528 init / pickIndex | $O(n)$ / $O(\log n)$ | $O(n)$ |

- **Killer gotchas:**

1. **Empty-prefix base case** — always init `freq[0]=1` (p560) or `seen[0]=-1` (p525/p523). Missing this causes every subarray starting at index 0 to be uncounted.
2. **p523: first-occurrence only** — never overwrite `mod_idx[prefix]`; the first index gives the longest possible window. Also require `i - mod_idx[prefix] > 1` (length 2).
3. **p238 assign-before-multiply on BOTH passes** — `output[i] = left` then `left *= nums[i]` on the left pass; `output[i] *= right` then `right *= nums[i]` on the right pass.
4. **p713 edge case** — if `k <= 1`: return 0 immediately (all positive nums have product 1).
5. **p528 1-indexed random** — `random.randint(1, total)` not `(0, total-1)`. You are picking a position in `[1, total]` weight-space, then finding which bucket it falls in.

Problem	Pattern	Key trick
p0560 Subarray Sum Equals K	prefix freq map	<code>freq[curr-k]</code> count; init <code>freq[0]=1</code>
p0238 Product Except Self	left+right product passes	assign <code>output[i]</code> BEFORE multiplying; $O(1)$ extra

Problem	Pattern	Key trick
p0525 Contiguous Array	0→-1 balance, first_seen map	<code>seen={0:-1}</code> ; same balance at $i,j \rightarrow$ equal 0s/1s in $(i,j]$
p0523 Continuous Subarray Sum	prefix mod, first_seen map	<code>mod_idx={0:-1}</code> ; gap>1 for len 2; never overwrite first index
p0713 Subarray Product < K	sliding window (not prefix!)	<code>k<=1→0</code> ; count += <code>right-left+1</code> subarrays ending at right
p0528 Random Pick with Weight	prefix sums + binary search	<code>randint(1,total)</code> (1-indexed); find first prefix t

randomized

- **Mindset:** Generate from a **uniform source** by mapping samples onto a grid and **rejecting the overhang** so every accepted outcome is equiprobable. For “random pick without repeat”, use **virtual Fisher-Yates**: swap the picked slot with the last slot in a lazy dict, then shrink the space — no actual array needed.
- **When to use:** “implement randX from randY”, random pick without repeats, shuffle, reservoir sampling — must be strictly uniform.

Skeleton — p470: Rand10 from Rand7 (Rejection Sampling)

```
def rand10():
    while True:
        row = rand7()          # 1..7
        col = rand7()          # 1..7
        idx = (row - 1) * 7 + col # 1..49 (uniform 7×7 grid)
        if idx <= 40:           # accept only 1..40 (perfect multiple of 10)
            return (idx - 1) % 10 + 1 # map to 1..10
        # reject 41..49 and retry ← the overhang
```

Why 40? $40 = 4 \times 10$, the largest multiple of 10 < 49 . Any bias-free range must be a multiple of the target range.

Skeleton — p519: Random Flip Matrix (Virtual Fisher-Yates)

```
class Solution:
    def __init__(self, m, n):
        self.m, self.n = m, n
        self.total = m * n          # number of unflipped cells
        self.mapping: dict[int,int] = {} # sparse remapping table

    def flip(self):
        r = random.randint(0, self.total - 1) # pick a random flat index
        x = self.mapping.get(r, r)             # resolve: what cell does r map to?
        last = self.total - 1
        last_val = self.mapping.get(last, last)
        self.mapping[r] = last_val              # swap r ← last (lazy)
        # only delete mapping[last] if last != r (don't erase what we just wrote)
        if last in self.mapping and last != r:
```

```

        del self.mapping[last]
    self.total -= 1          # shrink the space
    return [x // self.n, x % self.n]  # decode flat index → (row, col)

def reset(self):
    self.mapping = {}        # drop all remapping entries
    self.total = self.m * self.n  # restore full size

```

Key insight: The dict never stores cells at their “natural” position — it only stores redirects for cells that were swapped away. `mapping.get(k, k)` elegantly handles both cases: swapped and unswapped.

- **Complexity:**

Problem	Time per call	Space
p470 Rand10 from Rand7	Expected $O(1)$; ~ 1.225 <code>rand7()</code> calls per result (49/40 ratio)	$O(1)$
p519 flip	Expected $O(1)$ amortized	$O(\text{flips})$ for mapping dict
p519 reset	$O(\text{flips})$ to clear dict	$O(1)$ after clear

- **Killer gotchas:**

1. **Uniformity breaks without rejection:** `(rand7()+rand7())%10` is **biased** — sums are not uniform. Always use the 2D grid + reject-overhang technique.
2. **Overhang must be a multiple of target range:** Accept 40 (not 42 or 45). Any other cutoff introduces bias.
3. **p519 — stale last key:** After swapping `r ← last`, you must delete `mapping[last]` **unless** `r == last` (picked the last slot). Skipping the `last != r` guard leaves a phantom redirect that causes repeats.
4. **p519 — forgetting total -= 1:** Without shrinking the space, the same cell can be returned twice.
5. **p519 — decode flat index:** `x // n` gives row, `x % n` gives col. Easy to swap or omit.
6. **Judge determinism:** Both files use `random.seed(42)` for reproducible WASM judge output — keep it in both problem and solution files.

Problem	Key trick	Essence
p470 Rand10 from Rand7	7×7 grid $\rightarrow 1..49$; accept 40; <code>(idx-1)%10+1</code>	Cutoff = largest multiple of target source range
p519 Random Flip Matrix	Dict-based lazy swap-with-last; <code>mapping.get(k, k)</code>	Virtual Fisher-Yates; <code>r==last</code> edge case; flat $\rightarrow$ (row,col) decode

Skeleton — p470: Rand10 from Rand7 (Rejection Sampling)

stack

- **Mindset:** Push state so the most-recently-seen item is handled next — natural fit for nesting/matching, “what was I doing before?”, monotonic sequences, and expression eval.
- **When to use:** Bracket matching/nesting, decode/compress, monotonic (next greater/smaller), LIFO/recent-item tracking, expression evaluation, iterative DFS.

- **Skeleton:**

p020 — Bracket matching

```
mapping = {'(': '(', ')': ')', '[': '[', ']' : ']', '{': '{', '}' : '}'
stack = []
for ch in s:
    if ch in mapping:
        # closer
        if not stack or stack[-1] != mapping[ch]:
            return False
            # empty mid-loop OR mismatch
        stack.pop()
    else:
        stack.append(ch)
        # push opener
return len(stack) == 0
# unclosed openers?
```

p155 — Min Stack (running-min trick)

```
stack, min_stack = [], []
# push:
stack.append(val)
min_stack.append(val if not min_stack else min(val, min_stack[-1])) # global min so far
# pop:
stack.pop(); min_stack.pop()
# always in sync
# getMin: min_stack[-1]
# O(1) - top IS current global min
```

p394 — Decode String (nested repeat)

```
stack = [] # list of (repeat_count, string_before_bracket)
curr_num, curr_str = 0, ""
for ch in s:
    if ch.isdigit():
        curr_num = curr_num * 10 + int(ch) # multi-digit accumulate
    elif ch == '[':
        stack.append((curr_num, curr_str))
        curr_num, curr_str = 0, "" # RESET BOTH on every [
    elif ch == ']':
        repeat, prefix = stack.pop()
        curr_str = prefix + curr_str * repeat # prefix FIRST, then repeated segment
    else:
        curr_str += ch
return curr_str
```

- **Complexity:** $O(n)$ time, $O(n)$ space for all three.

- **Killer gotchas:**

- **p020:** Two separate failure conditions — don't collapse them. `not stack` and `stack[-1] != mapping[ch]` both matter.
- **p155:** `min_stack` stores the **running global minimum**, not just the pushed value. `min_stack[-1]` after any push = minimum of the entire current stack. Always push/pop both stacks together or `getMin` drifts.
- **p394:** Reset **both** `curr_num = 0` and `curr_str = ""` on `[`. Forgetting `curr_num` reset bleeds digit state across nesting levels. Reconstruction order is `prefix + curr_str * repeat` (not reversed).

Problem	Difficulty	Key trick	Essence
p020 Valid Parentheses	Easy	close→open map + two failure modes	Push openers; closer checks not stack OR top mapping[ch]; return stack==[] at end
p155 Min Stack	Medium	min_stack stores running global min	min_stack.append(min(val, min_stack[-1])); both stacks pop together → O(1) getMin
p394 Decode String	Medium	Multi-digit accumulate + reset BOTH vars on [	Push (curr_num, curr_str) on [; on]: prefix + curr_str * repeat; reset num AND str

top_k_elements

- **Mindset:** Keep a heap of bounded size k — a **min-heap of size k retains the k largest** (evicts the smallest), a **max-heap of size k retains the k smallest** (evicts the largest). After processing all n elements, the root is your answer. Python's `heapq` is min-heap only; negate keys to simulate a max-heap.
- **When to use:** “ k th largest/smallest”, “top/bottom k ”, “ k closest / most frequent”, streaming top- k where full $O(n \log n)$ sort is wasteful. Also: 2D elevation problems solvable with Dijkstra-like min-heap BFS from a boundary.
- **Skeleton:**

```
import heapq
from collections import Counter

# --- Pattern A: k LARGEST elements (min-heap of size k) ---
def kth_largest(nums, k):
    heap = []
    for x in nums:
        heapq.heappush(heap, x)          # min at root
        if len(heap) > k:
            heapq.heappop(heap)         # evict smallest → keeps k largest
    return heap[0]                     # root = kth largest

# Shortcut (non-streaming): heapq.nlargest(k, nums)[-1]
# Shortcut (non-streaming): heapq.nsmallest(k, nums)[-1]

# --- Pattern B: k CLOSEST (max-heap of size k, negate distance) ---
def k_closest(points, k):
    heap = []
    for x, y in points:
        dist = -(x*x + y*y)            # negate → min-heap acts as max-heap
        heapq.heappush(heap, (dist, [x, y]))
        if len(heap) > k:
```

```

        heapq.heappop(heap)          # evict farthest
    return [p for _, p in heap]

# --- Pattern C: k MOST FREQUENT (Counter + sort/heap) ---
def top_k_frequent(nums, k):
    count = Counter(nums)
    # Sort approach O(n log n): good enough for most interviews
    return [item for item, _ in
            sorted(count.items(), key=lambda x: -x[1])[:k]]
    # Heap approach O(n log k):
    # return [item for item, _ in heapq.nlargest(k, count.items(), key=lambda x: x[1])]
    # Bucket sort O(n): buckets[freq].append(item); scan from n down

# --- Pattern D: 2D Min-Heap BFS (Trapping Rain Water II) ---
def trap_rain_water_ii(heightMap):
    m, n = len(heightMap), len(heightMap[0])
    visited = [[False]*n for _ in range(m)]
    heap = []                                # min-heap by cell height
    # Seed with all boundary cells
    for r in range(m):
        for c in [0, n-1]:
            heapq.heappush(heap, (heightMap[r][c], r, c))
            visited[r][c] = True
    for c in range(1, n-1):
        for r in [0, m-1]:
            heapq.heappush(heap, (heightMap[r][c], r, c))
            visited[r][c] = True
    water = 0
    for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
        ... # always pop min-height boundary, water += max(0, h - neighbor_h)
            # push max(h, neighbor_h) to maintain water level
    return water

```

- **Complexity:**

Approach	Time	Space
Heap of size k	$O(n \log k)$	$O(k)$
<code>heapq.nlargest/nsmallest</code>	$O(n \log k)$	$O(k)$
Full sort	$O(n \log n)$	$O(1)$
Quickselect	$O(n)$ avg, $O(n^2)$ worst	$O(1)$
Bucket sort (p347)	$O(n)$	$O(n)$
2D BFS min-heap (p407)	$O(mn \log(mn))$	$O(mn)$

- **Killer gotchas:**

1. **Heap direction is backwards from intuition:** min-heap of size k keeps the k LARGEST (evicts the small ones). Want k smallest? Use max-heap (negate).
2. **p215 solution uses `heapq.nlargest(k, nums)[-1]`** — not a manual push/pop loop. Both are valid; the manual loop is better for streaming/memory-constrained contexts.
3. **p347 solution uses `sorted()`, not a heap** — Counter + sort is $O(n \log n)$ but simpler to write. The $O(n \log k)$ heap or $O(n)$ bucket sort are follow-up optimisations.
4. **p407 uses a MIN-heap (not max)** — it's Dijkstra from the boundary inward. Always pop the lowest wall first; water trapped = `max(0, popped_h - neighbor_h)`; push

`max(popped_h, neighbor_h)` to carry the “current water level” forward. Requires a `visited` grid to avoid revisiting cells.

5. **p973: skip sqrt** — compare dist^2 directly (monotone transformation preserves order).

Problem	Difficulty	Heap type	Key trick
p215 Kth Largest	Medium	min-heap size k	<code>heapq.nlargest(k, nums)[-1]</code> or manual push+pop; root = answer
p347 Top K Frequent	Medium	sort (or heap)	Counter → sort by -freq ; bucket-sort for $O(n)$ follow-up
p973 K Closest Points	Medium	max-heap size k	negate dist^2 (skip sqrt); push $(-\text{dist}^2, \text{point})$, pop when $\text{len} > k$
p407 Trapping Rain Water II	Hard	min-heap BFS	Dijkstra from boundary; push <code>max(wall_h, nbr_h)</code> to carry water level; needs <code>visited</code> grid

two_heaps

- **Mindset:** Maintain two heaps — **small** (max-heap, lower half) and **large** (min-heap, upper half) — so the partition boundary is always at one of the two roots. Every insert goes through **small** first, then gets routed to maintain sorted order between heaps.
- **When to use:** “median of a data stream/window”, split elements into two balanced halves, k-way merge of sorted feeds, need $O(1)$ access to both ends of an ordered partition.
- **Skeleton (p295 — median of stream):**

```
import heapq
small, large = [], [] # small = max-heap (negate), large = min-heap

def add(num):
    heapq.heappush(small, -num) # 1. always push to small
    heapq.heappush(large, -heapq.heappop(small)) # 2. move small's top → large (ensures order)
    if len(large) > len(small): # 3. rebalance: invariant = small {large, large+1}
        heapq.heappush(small, -heapq.heappop(large))

def median():
    if len(small) > len(large):
        return float(-small[0])
    return (-small[0] + large[0]) / 2.0
```

- **Skeleton (p480 — sliding window median, lazy deletion):**

```
small: list[tuple[int,int]] = [] # (-val, idx) - max-heap
large: list[tuple[int,int]] = [] # ( val, idx) - min-heap
deleted: dict[int,int] = {} # idx → pending deletion count
```

```
in_small: dict[int, bool] = {}      # idx → True if in small, False if in large
small_size = large_size = 0        # LOGICAL sizes (heap may hold stale entries)

def prune(heap):
    while heap and heap[0][1] in delayed:
        idx = heap[0][1]
        delayed[idx] -= 1
        if delayed[idx] == 0: del delayed[idx]
        heapq.heappop(heap)

def make_balanced():
    # nonlocal small_size, large_size
    if small_size > large_size + 1:
        prune(small); val, idx = heapq.heappop(small)
        heapq.heappush(large, (-val, idx)); in_small[idx] = False
        small_size -= 1; large_size += 1; prune(small)
    elif small_size < large_size:
        prune(large); val, idx = heapq.heappop(large)
        heapq.heappush(small, (-val, idx)); in_small[idx] = True
        large_size -= 1; small_size += 1; prune(large)

for i, num in enumerate(nums):
    prune(small); prune(large)          # proactive: clear stale tops before insert
    # Insert: go to small if current small-root, else large
    if not small or num <= -small[0][0]:
        heapq.heappush(small, (-num, i)); in_small[i] = True; small_size += 1
    else:
        heapq.heappush(large, (num, i)); in_small[i] = False; large_size += 1
    make_balanced()

    if i >= k:                          # evict element leaving window
        out = i - k
        delayed[out] = delayed.get(out, 0) + 1
        if in_small.get(out, True): small_size -= 1
        else: large_size -= 1
        make_balanced()

    if i >= k - 1:
        prune(small); prune(large)
        result.append(-small[0][0] if k%2==1 else (-small[0][0]+large[0][0])/2.0)
```

- **Skeleton (p355 — design Twitter, heap merge):**

```
# Store tweets as (-timestamp, tweetId) per user; use global counter for ordering
class Twitter:
    def __init__(self):
        self.ts = 0
        self.tweets = {}  # userId → [(-ts, tweetId), ...]
        self.following = {}  # followerId → set of followeeIds

    def postTweet(self, userId, tweetId):
        self.tweets.setdefault(userId, []).append((-self.ts, tweetId))
```

```

self.ts += 1

def getNewsFeed(self, userId):
    # Dump ALL tweets from self + followees into heap, pop top 10
    # O(T log T) where T = total tweets of all followed users
    heap = []
    for uid in self.following.get(userId, set()) | {userId}:
        for tweet in self.tweets.get(uid, []):
            heapq.heappush(heap, tweet)
    return [heapq.heappop(heap)[1] for _ in range(min(10, len(heap)))]

```

- **Complexity:** | Problem | Time | Space | |—|—|—| | p295 add/findMedian | $O(\log n)$ / $O(1)$ | $O(n)$ | | p480 per window step | $O(\log n)$ amortized | $O(n)$ | | p355 getNewsFeed | $O(T \log T)$ where T = total followee tweets | $O(T)$ |
- **Killer gotchas:**
 1. **Heap ordering guarantee (p295):** The push-to-small $\rightarrow$ pop-to-large sequence is what guarantees every element in **large** every element in **small**. Skipping this and inserting directly breaks the invariant silently.
 2. **Lazy deletion requires index, not value (p480):** Duplicate values make value-based deletion impossible. Always store (**val**, **idx**) tuples so each element is uniquely identifiable.
 3. **in_small is mandatory (p480):** When evicting a window element, you must know which heap it logically belongs to in order to decrement the correct size counter. The heap may not even have it at its root (it's stale); the size counter must be updated without touching the heap.
 4. **delayed decrements, not deletes (p480):** The same index can theoretically be marked multiple times (guard against accidents). Always **delayed[idx] -= 1** and only **del** when it hits 0.
 5. **Prune timing matters (p480):** Call **prune** proactively at loop start, after **make_balanced()**, AND before reading the median. Failing to prune before reading roots yields stale median values.
 6. **p355 is NOT a k-way pointer merge:** The solution pushes all tweets onto a single heap — simple but $O(T \log T)$. A pointer-per-user approach would be $O(10 \log U)$ per call.

Problem	Difficulty	Key trick	Essence
p295 Median Finder	Hard	push $\rightarrow$ move $\rightarrow$ rebalance	Max-heap lower half + min-heap upper half; invariant: $\backslash \text{small} \backslash \{ \text{large}, \text{large}+1 \}$
p480 Sliding Window Median	Hard	lazy deletion via index tuples + in_small size tracking	Two heaps + delayed dict; decouple logical size from heap physical size
p355 Design Twitter	Medium	negative timestamp for max-heap ordering	Dump all followee tweets into heap; negate timestamp to get newest-first order

backtracking

- **Mindset:** Build a partial solution choice-by-choice; the moment a branch violates constraints or cannot exceed the best known solution, abandon (back-track) it. Aggressive pruning on a decision tree.
- **When to use:** “find all **valid** combinations/partitions satisfying a constraint”, “return true if possible to form/partition”, small N (~20), constraint satisfaction. Signals: *find all combinations that sum to, partition into equal, matchsticks to square.*
- **Skeleton (Standard start-index for subsets/combinations):**

```
def solve(candidates, target):
    result = []
    candidates.sort()          # sorting is essential for early pruning!

    def backtrack(start: int, remaining: int, path: list[int]):
        if remaining == 0:
            result.append(path[:])  # clone the path (shallow copy)
            return
        for i in range(start, len(candidates)):
            # Pruning: since candidates are sorted, if this one is too big, all later ones are
            if candidates[i] > remaining:
                break

            # Choose
            path.append(candidates[i])
            # Explore (start index = i allows reusing same element; i+1 forbids reuse)
            backtrack(i, remaining - candidates[i], path)
            # Un-choose
            path.pop()

    backtrack(0, target, [])
    return result
```

- **Skeleton (Subset sum/partition to K equal buckets — e.g. p473):**

```
def makesquare(matchsticks: list[int]) -> bool:
    total = sum(matchsticks)
    if total % 4 != 0: return False
    side = total // 4
    matchsticks.sort(reverse=True)  # BIG matchsticks first → prunes invalid paths immediately
    if matchsticks[0] > side: return False
    sides = [0] * 4

    def backtrack(idx: int) -> bool:
        if idx == len(matchsticks):
            return sides[0] == sides[1] == sides[2] == side
        for i in range(4):
            if sides[i] + matchsticks[idx] <= side:
                sides[i] += matchsticks[idx]  # Choose
                if backtrack(idx + 1):          # Explore
                    return True
                sides[i] -= matchsticks[idx]  # Un-choose
        return False

    return backtrack(0)
```

```
return backtrack(0)
```

- **Complexity:** $O(2^N)$ subset-style to $O(N!)$ permutation-style worst case (assignment to buckets = $O(k^N)$). Pruned branches reduce search space dramatically. $O(N)$ space for recursion stack.
- **Killer gotchas:**
 1. **Forgetting `path.pop()` or `sides[i] -= val`** — leaks state into sibling branches, corrupting the search.
 2. **Pruning without sorting** — for p039, if candidates are unsorted, you cannot **break** the loop early when `candidates[i] > remaining` because a smaller valid candidate might appear later.
 3. **Zuma Game (p488) memo key** — needs immutable types. Use `(board_str, hand_sorted_tuple)` as the memoization dictionary key.
 4. **Per-level deduplication (p491)** — when duplicates exist in the input but we cannot sort it (as sorting would break the subsequence order), use a local `used = set()` *inside* each `backtrack()` call to prevent processing the same value multiple times at the same level.
 5. **Sort descending for bucket-fill** — in partition problems (p473), filling bins with the largest items first causes early capacity overflows, fanning out fewer recursion branches.

Problem	Essence	Key trick
p017 Letter Combinations	Digit-to-letters combinations	Standard DFS; one char added per recursion depth
p039 Combination Sum	Combinations with unlimited reuse	Sort candidates + break loop when <code>num > remaining</code> ; reuse index <code>i</code> in recursive call
p078 Subsets	Power set of unique elements	Append <code>path[:]</code> to result at the start of each recursive call (records all prefixes)
p473 Matchsticks to Square	Partition array into 4 equal sum subsets	Sort descending, backtrack by placing matchstick <code>idx</code> into 1 of 4 sides
p491 Non-decreasing Subseqs	Enumerate non-decreasing subsequences	Keep input order; use a local <code>used = set()</code> at each level to dedup sibling calls
p488 Zuma Game	Min balls to clear board	DFS + memoization on <code>(collapsed_board, hand_tuple)</code>

cyclic_sort

- **Mindset:** When values map ~1:1 onto indices, use the array as its own hash table. Two strategies: (1) **Cyclic-sort swap** — physically place each number at its “correct” index, then scan for mismatches. (2) **Negation-mark** — flip the sign at `abs(val)-1` to mark “seen”; a second pass reads off violations.
- **When to use:** Array of `n` numbers in range `[0,n]` or `[1,n]`; asked to find missing / duplicate / disappeared numbers in **$O(n)$ time + $O(1)$ space**. Signal: range-of-values == array-length.
- **Skeleton:**

```
# --- Strategy 1: Cyclic-sort swap (p268 pattern, range [0,n]) ---
i = 0
```

```

while i < n:
    correct = nums[i]          # where nums[i] SHOULD live
    if correct < n and nums[i] != nums[correct]:
        nums[i], nums[correct] = nums[correct], nums[i]    # swap into place
    else:
        i += 1                # only advance when nothing useful to swap

# After sort: first index where nums[i] != i is the missing value; if none → n
for i in range(n):
    if nums[i] != i:
        return i
return n

# --- Strategy 2: Negation-mark (p442/p448 pattern, range [1,n]) ---
for num in nums:
    idx = abs(num) - 1        # must use abs() - value may already be negated
    if nums[idx] < 0:
        result.append(abs(num)) # already seen → duplicate (p442)
    else:
        nums[idx] = -nums[idx] # mark visited

# Find disappeared: indices still positive after pass
return [i + 1 for i in range(n) if nums[i] > 0] # (p448)

```

- **Complexity:** $O(n)$ time, $O(1)$ space.

- Cyclic-sort: each element swaps at most once → amortized $O(1)$ per element.
- Negation-mark: single pass + single scan.

- **Killer gotchas:**

1. **correct < n guard (p268 only).** Range is $[0, n]$ so value n has no valid home (index n is out-of-bounds). Without this guard → index error or infinite swap loop.
2. **Always read with abs(num).** Once you start negating, raw `num` is corrupted mid-pass. Every index computation must be `abs(num) - 1`.
3. **Advance i only in the else branch** (cyclic-sort). If you always do `i += 1` you'll skip elements that just landed at `i` via a swap.
4. **What to append in p442.** Append `abs(num)` (the current value), NOT `idx + 1`. They're equal for valid $[1, n]$ input, but using `abs(num)` is semantically correct and safe when you've already negated prior elements.
5. **p442 vs p448 post-pass logic is opposite:** p442 collects values that *land on* an already-negative index; p448 collects indices that are *still positive* after the pass.

Problem	Range	Algorithm	Key trick
p268 Missing Number	$[0, n]$	Cyclic-sort swap	Guard <code>correct < n</code> ; after sort, first <code>nums[i]</code> $i \rightarrow$ answer; fallback <code>n</code>
p442 Find Duplicates	$[1, n]$	Negation-mark	<code>idx=abs(num)-1</code> ; already-negative → append <code>abs(num)</code>
p448 Find Disappeared	$[1, n]$	Negation-mark	Same negate pass; collect <code>i+1</code> where <code>nums[i] > 0</code> after pass

modified_binary_search

- **Mindset:** When the search space is monotonic but not a plain sorted array, halve it via a predicate — by *discovering which half is sorted* (rotated array) or by binary-searching the **answer range** itself (feasibility / “minimize the max”).
- **When to use:** $O(\log n)$ on a rotated/bi-tonic array **OR** “minimize the max / maximize the min” where feasibility is monotonic. Signals: *rotated sorted array, koko / min speed, split array largest sum*, “smallest value such that...”.
- **Skeleton:**

Pattern A — half-is-sorted (rotated array search):

```
# Target may not exist → while lo <= hi, return -1
lo, hi = 0, len(nums) - 1
while lo <= hi:
    mid = lo + (hi - lo) // 2
    if nums[mid] == target:
        return mid
    if nums[lo] <= nums[mid]:          # left half is sorted
        if nums[lo] <= target < nums[mid]:
            hi = mid - 1              # target in left half
        else:
            lo = mid + 1              # target in right half
    else:                             # right half is sorted
        if nums[mid] < target <= nums[hi]:
            lo = mid + 1              # target in right half
        else:
            hi = mid - 1              # target in left half
return -1
```

Pattern B — answer-space (minimize feasible value):

```
# Answer always exists → while lo < hi, return lo
lo, hi = min_possible, max_possible
while lo < hi:
    mid = lo + (hi - lo) // 2
    if feasible(mid):
        hi = mid                    # mid is valid; keep it, try smaller
    else:
        lo = mid + 1                # mid too small; discard it
return lo                          # lo == hi == minimum feasible value

# feasible() must be MONOTONIC: once True stays True as value grows
# --- p875 Koko ---
def feasible(k):
    return sum(math.ceil(p / k) for p in piles) <= h

# --- p410 Split Array ---
def feasible(max_sum):
    count, cur = 1, 0
```

```

for n in nums:
    cur += n
    if cur > max_sum:    # overflow → new subarray
        count += 1
        cur = n
    if count > k: return False
return True

```

- **Complexity:** A: $O(\log n)$. B: $O(n \cdot \log M)$, where M = answer-range width.
- **Killer gotchas:**
 1. **Loop difference is critical:** Pattern A uses `while lo <= hi` (target may be absent); Pattern B uses `while lo < hi` (answer always exists). Mixing them causes off-by-one bugs.
 2. **Pattern B: use `hi = mid`, not `hi = mid - 1`** — `mid` is feasible so it's a valid candidate; never discard it.
 3. **Pattern A: use `nums[lo] <= nums[mid]` (with `<=`)** — handles 2-element arrays where `lo == mid` (left half is trivially sorted).
 4. **Pattern A has two symmetric branches** — both need the “in sorted half?” range check and both else-branches redirect to the opposite half.
 5. **Pattern B: `feasible()` must be monotonic** — if `feasible(x)` is True, then `feasible(x+1)` must also be True. Verify this before applying the template.
 6. **Pattern B answer-range bounds matter:** Koko: `[1, max(piles)]`; Split Array: `[max(nums), sum(nums)]` — getting these wrong silently produces wrong answers.
- **How it differs from plain BS:** Plain BS compares `nums[mid]` to a `target` in a clean sorted array. Modified BS either *discovers structure first* (which half is sorted) before narrowing, or *abandons the array entirely* — searching a numeric **answer** via `feasible(mid)`, never comparing array values to a target.

Problem	Pattern	Answer range	Key trick
p033 Search Rotated Sorted Array	A — rotated	index <code>[0, n-1]</code>	<code>nums[lo] <= nums[mid]</code> → left sorted; check if target inside <code>[lo, mid)</code> , else go right
p875 Koko Eating Bananas	B — minimize	<code>[1, max(piles)]</code>	<code>feasible(k): sum(pile/k) ≤ h</code> ; as <code>k</code> grows, hours decrease → monotonic
p410 Split Array Largest Sum	B — minimize	<code>[max(nums), sum(nums)]</code>	<code>feasible(cap):</code> greedy pack into subarrays <code>cap</code> ; count subarrays <code>k</code>

subsets

- **Mindset:** Systematically generate **every** member of a combinatorial space (subsets / combinations / permutations) by building partial paths and recording results — pure enumeration, minimal pruning. The space is the answer itself.
- **When to use:** “return **all** subsets / combinations / permutations / power set”, small n (~ 20), no optimization phrasing. Signal: the word *all* + a structure (subset, combo, perm). If there’s a goal constraint (`sum == target`, `is-valid?`), use backtracking.

- **Complexity:** Subsets $O(n \cdot 2)$; Combinations $O(k \cdot C(n,k))$; Permutations $O(n \cdot n!)$.
 - **Killer gotcha: Duplicate subsets** with repeats (p090) — sort first, then skip if `i > start` and `nums[i] == nums[i-1]`. The `i > start` (not `i > 0`) is essential: dedup only within one loop level, still allowing the same value deeper in the tree.
-

Skeleton A — Subsets / Combinations (start-index, choose→explore→unchoose):

```
def backtrack(start, path):
    result.append(path[:])          # Subsets: record every node
    # if len(path) == k: result.append(path[:]); return # Combinations: record leaf only

    for i in range(start, len(nums)):
        # Pruning (Combinations only): stop if not enough elements remain
        # if (len(nums) - i + 1) < (k - len(path)): break

        # Dedup (Subsets II only): skip same value at same level
        # if i > start and nums[i] == nums[i-1]: continue

        path.append(nums[i])
        backtrack(i + 1, path)      # i+1 prevents reuse
        path.pop()

    backtrack(0, [])
```

Skeleton B — Permutations (iterative BFS insertion — actual p046 approach):

```
perms = [[]]
for num in nums:
    new_perms = []
    for p in perms:
        for i in range(len(p) + 1):    # insert num at every position
            new_perms.append(p[:i] + [num] + p[i:])
    perms = new_perms
return sorted(perms)
```

Skeleton B-alt — Permutations (recursive DFS with used set):

```
def backtrack(path, used):
    if len(path) == len(nums):
        result.append(path[:])
        return
    for i in range(len(nums)):
        if i in used: continue
        used.add(i)
        path.append(nums[i])
        backtrack(path, used)
        path.pop()
        used.discard(i)

backtrack([], set())
```

vs backtracking: identical skeleton, opposite philosophy.

Subsets records **every** partial path, rarely prunes → *enumerate all*.

Backtracking records only **goal-reaching** paths, prunes hard → *search for the valid*.

Problem	Essence	Key Trick
p046 Permutations	All orderings of distinct ints	Iterative insertion: insert new num at every position of each existing perm
p077 Combinations	All k-sized subsets from [1..n]	start index prevents reuse; record only at len(path)==k ; can prune loop when n-i+1 < k-len(path)
p090 Subsets II	Power set with duplicate elements	Sort + per-level dedup: if i > start and nums[i] == nums[i-1] : continue

trie

- **Mindset:** Store strings by shared character prefixes in a tree — insert/search/ prefix-lookup is O(word length), common prefixes share nodes (memory efficient for large word sets). Two flavors of end-node: **is_end**: bool for exact-word queries; **word**: str | None for grid-DFS where you need the word itself at collection time.
- **When to use:** prefix matching, autocomplete, dictionary with wildcard ., word search on a grid, “words formed by concatenating others”. Signals: many strings + repeated **prefix** queries, or board-DFS that must early-abort non-prefix paths.
- **Skeleton:**

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.is_end = False          # or: self.word: str | None = None (grid DFS)

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def _find(self, prefix: str) -> TrieNode | None:
        """Shared helper - avoids duplicating walk logic."""
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return None
```

```
        node = node.children[ch]
    return node

def search(self, word: str) -> bool:
    node = self._find(word)
    return node is not None and node.is_end # must check is_end!

def starts_with(self, prefix: str) -> bool:
    return self._find(prefix) is not None # reachability only

# Wildcard '.' - DFS with index (p211):
def _dfs(self, node, word, index):
    if index == len(word):
        return node.is_end
    ch = word[index]
    if ch == '.':
        return any(self._dfs(child, word, index+1)
                    for child in node.children.values())
    if ch not in node.children:
        return False
    return self._dfs(node.children[ch], word, index+1)

# Grid DFS along trie edges (p212) - store word at leaf, mark visited in-place:
def dfs(r, c, parent):
    ch = board[r][c]
    curr = parent.children.get(ch)
    if not curr: return
    if curr.word: # found a complete word
        result.append(curr.word)
        curr.word = None # deduplicate: null out immediately
    board[r][c] = '#' # mark visited (in-place, no visited set)
    for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
        nr, nc = r+dr, c+dc
        if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] != '#':
            dfs(nr, nc, curr)
    board[r][c] = ch # restore
    if not curr.children: # leaf pruning: remove dead branch
        parent.children.pop(ch)

# Concatenated words (p472) - trie + recursive DFS, insert AFTER check:
words_sorted = sorted(words, key=len)
for word in words_sorted:
    if can_concatenate(word, start=0, count=0):
        result.append(word)
    insert(word) # insert AFTER check - prevents self-match!

def can_concatenate(word, start, count):
    node = root
    for i in range(start, len(word)):
        ch = word[i]
        if ch not in node.children: return False
        node = node.children[ch]
```



```

    if node.is_end:
        if i == len(word) - 1: return count >= 1 # 2 pieces total
        if can_concatenate(word, i+1, count+1): return True
    return False

```

- **Complexity:** Insert/search $O(L)$. Wildcard search $O(\Sigma^D \cdot L)$ worst ($D = \#$ of dots). Grid $O(m \cdot n \cdot 4^L)$ worst but trie-pruned in practice. Space $O(\Sigma \cdot N)$ for trie nodes ($\Sigma =$ alphabet size, $N =$ total characters across all words).
- **Killer gotchas:**
 1. **search vs starts_with** — `starts_with("app")` returns True even if only “apple” is inserted; `search("app")` must check `node.is_end`. Never skip the `is_end` check for exact-word queries.
 2. **p212 — two-level pruning required for no TLE:**
 - Null `curr.word = None` after collecting (prevents re-adding duplicates)
 - Pop `parent.children[ch]` when not `curr.children` (removes exhausted branches, shrinks the trie as the DFS progresses — critical for large inputs)
 3. **p212 — pass parent, not curr, into DFS** so that leaf pruning (`parent.children.pop(ch)`) is possible without an extra reference.
 4. **p472 — sort by length + insert AFTER check.** If you insert all words first, a word matches itself (1 piece = itself), giving a false positive. Sorting ensures you only check against strictly shorter words already in the trie.
 5. **p472 — count >= 1 not count >= 2** at the terminal check: `count` is incremented when you *start* a new piece; the last piece adds to `count` only if `can_concatenate` recurses again, so at the final character `count >= 1` means 2 total pieces.

Problem	Essence	Key trick
p208 Implement Trie	Foundation: insert/search/starts_with	<code>_find()</code> helper shared by both; search checks <code>node.is_end</code> , starts_with checks reachability only
p211 Design Add/Search Words	Trie + wildcard .	DFS with index <code>_dfs(node, word, idx)</code> ; at ., iterate all <code>node.children.values()</code>
p212 Word Search II	Build trie from words, DFS board along trie edges	Store word at leaf (not <code>is_end</code>); in-place '#' marking; prune: null word + pop empty children
p472 Concatenated Words	Trie + recursive DFS to split word into 2 pieces	Sort by length, insert word AFTER check; <code>count >= 1</code> at end = 2 pieces total

graph

- **Mindset:** Model relationships as nodes+edges. Two sub-patterns:
 - (1) **Topological sort (Kahn’s BFS)** — for dependency/ordering with cycle detection;
 - (2) **Net in/out-degree score** — when you just need to find a “special node” with no traversal needed. DFS-based topo sort is equally valid but Kahn’s is interview-safe.
- **When to use:**
 - Prerequisites / dependencies / ordering → **topological sort** (Kahn’s BFS)

– “Trusted by all / trusts nobody” → **net degree score** (one pass, no graph)

- **Skeleton (Kahn’s BFS — covers p207 & p210):**

```
from collections import deque

def kahn(n, edges):          # edges: list of [dest, src] (dest depends on src)
    graph = {i: [] for i in range(n)}    # src → [dest, ...]
    in_degree = [0] * n
    for dest, src in edges:
        graph[src].append(dest)
        in_degree[dest] += 1

    queue = deque(i for i in range(n) if in_degree[i] == 0)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nei in graph[node]:
            in_degree[nei] -= 1
            if in_degree[nei] == 0:
                queue.append(nei)

    # p207: return len(order) == n          ← cycle detection only
    # p210: return order if len(order)==n else []    ← emit order
```

- **Skeleton (net-degree score — covers p997):**

```
def find_special(n, trust):    # nodes are 1-indexed → size n+1
    scores = [0] * (n + 1)    # net = in_degree - out_degree
    for a, b in trust:
        scores[a] -= 1        # a trusts someone → out_degree++
        scores[b] += 1        # b is trusted → in_degree++
    for i in range(1, n + 1):
        if scores[i] == n - 1: # trusted by all n-1, trusts nobody
            return i
    return -1
```

- **Complexity:** $O(V+E)$ time & space (both patterns).

- **Killer gotchas:**

1. **Edge direction:** Add edge `prereq → course`, bump `in_degree[course]`. Reversing it seeds the wrong zero-in-degree nodes and silently produces wrong answers.
2. **p997 nodes are 1-indexed (1..n):** `scores = [0] * (n+1)`, loop `range(1, n+1)`. Using size-`n` or 0-indexed is an off-by-one that’s easy to miss.
3. **p997 score == n-1, not n:** The judge is trusted by all OTHER `n-1` people (not themselves). Score `n` would be impossible.
4. **Cycle detection:** `len(order) == n` (not `> 0`). If even one node is stuck in a cycle its in-degree never reaches 0, so it’s never enqueued.
5. **DFS is equally valid** for topo sort (color-based: white/gray/black). Kahn’s is preferred in interviews because it’s iterative and cycle detection is a natural byproduct (no extra visited set needed).

Problem	Difficulty	Essence	Key trick
p207 Course Schedule	Medium	Can all courses be taken? (cycle detection)	Kahn's BFS; <code>len(order)==n → True</code>
p210 Course Schedule II	Medium	Return a valid course order, [] if cycle	Same Kahn's; collect <code>order</code> , return [] on cycle
p997 Find the Town Judge	Easy	Find node trusted by everyone, trusts nobody	Net score array <code>[0]*(n+1)</code> ; judge score <code>== n-1</code>

greedy

- **Mindset:** Sweep once carrying a single “best frontier” variable (`max_reach`, earliest `end`, running surplus) and update/reset on violations. A local optimum never blocks a better future.
- **When to use:** “can reach? / minimum count / valid starting point” over a sequence or intervals where a local invariant is monotonic and sufficient. Signals: *furthest jump*, *minimize arrows*, *cooldown task scheduling*, *maximum profit IPO*.
- **Skeleton (Interval selection / p452):**

```
def find_min_arrows(points: list[list[int]]) -> int:
    if not points: return 0
    # Always sort by END coordinate (right endpoint)
    points.sort(key=lambda x: x[1])
    arrows = 1
    end = points[0][1]
    for start, finish in points[1:]:
        if start > end:           # no overlap with current arrow reach
            arrows += 1
            end = finish         # place new arrow at the end of this balloon
    return arrows
```

- **Skeleton (Heap-based greedy selection / p502):**

```
import heapq

def find_max_capital(k: int, w: int, profits: list[int], capital: list[int]) -> int:
    projects = sorted(zip(capital, profits))
    max_heap = []
    i = 0
    for _ in range(k):
        # Push profits of all projects we can currently afford
        while i < len(projects) and projects[i][0] <= w:
            heapq.heappush(max_heap, -projects[i][1]) # negate for max-heap
            i += 1
        if not max_heap:
            break
        w += -heapq.heappop(max_heap) # pick project with max profit
    return w
```

- **Skeleton (Two-pass neighbor constraints / p135):**

```
def candy(ratings: list[int]) -> int:
    n = len(ratings)
    candies = [1] * n
    # Pass 1: left-to-right (ensure right-neighbor gets more if rating is higher)
    for i in range(1, n):
        if ratings[i] > ratings[i-1]:
            candies[i] = candies[i-1] + 1
    # Pass 2: right-to-left (ensure left-neighbor gets more if rating is higher)
    for i in range(n-2, -1, -1):
        if ratings[i] > ratings[i+1]:
            candies[i] = max(candies[i], candies[i+1] + 1)
    return sum(candies)
```

- **Complexity:** $O(N)$ for single-pass / multi-pass, $O(N \log N)$ if sorting or heaps are used. $O(1)$ extra space (or $O(N)$ for heaps/sorting).
- **Killer gotchas:**
 1. **Sorting intervals by start (p452)** — fails for nested intervals (e.g. $[1, 10]$ and $[2, 3]$). You must sort by **end** point to greedily resolve intervals that finish earliest.
 2. **Gas station (p134) local vs global** — if **current_surplus** drops below 0 at i , the starting point must be reset to $i + 1$. This is valid, but you **MUST** verify **total_surplus** ≥ 0 at the very end to guarantee a complete cycle is possible.
 3. **Task Scheduler (p621) idle slots** — formula is $(\text{max_freq} - 1) * (n + 1) + \text{max_count}$. The -1 is because the last execution of the most frequent task does not require cooldown after it.
 4. **Multi-machine task scheduler (p621b) simulation** — cooldown is per-machine. At each step, greedily assign task with highest remaining frequency. If stuck (no machine can execute any task), fast-forward **time** directly to the next cooldown expiry, avoiding empty loops.

Problem	Essence	Key trick
p055 Jump Game	Farthest reachable index	Update <code>max_reach = max(max_reach, i + nums[i])</code> ; return <code>False</code> if $i > \text{max_reach}$
p134 Gas Station	Find circular starting point	Reset start to $i+1$ when <code>current_surplus < 0</code> ; return start if <code>total_surplus >= 0</code> else <code>-1</code>
p135 Candy	Distribute candies based on ratings	Two passes: $L \rightarrow R$, then $R \rightarrow L$ taking <code>max(candies[i], candies[i+1]+1)</code>
p452 Arrows to Burst Balloons	Min arrows to pop all intervals	Sort by end coordinate; shoot arrow at end , increment when start $>$ end
p455 Assign Cookies	Satisfy children with cookies	Sort both; two-pointer match child greed vs cookie size
p502 IPO	Maximize capital using at most k projects	Sort by capital; max-heap of profits of currently affordable projects; pop k times
p621 Task Scheduler	Min CPU cycles with cooldown n	Formula: <code>max(len(tasks), (max_freq - 1) * (n + 1) + max_count)</code>

Problem	Essence	Key trick
p621b Task Scheduler Multi-Machine	Task scheduler with m machines	Simulation: assign highest-freq task not in cooldown per machine; fast-forward time on deadlock

matrix_traversal

- **Mindset:** Three sub-patterns: (1) *index math in-place* (transpose + reverse for rotation), (2) *boundary pointer simulation* (shrinking top/bottom/left/right for spiral), (3) *diagonal grouping by index sum* ($d = i+j$) for diagonal traversal, (4) *multi-source BFS* treating the grid as an implicit graph for distance problems.
- **When to use:** rotate/transform in-place $\rightarrow$ transpose + row-reverse; spiral/layer order $\rightarrow$ 4 boundary pointers + shrink-and-guard; diagonal order $\rightarrow$ group by $i+j$, reverse even diagonals; “nearest X per cell” $\rightarrow$ multi-source BFS from all X-cells simultaneously.
- **Skeleton:**

```
# --- p048: Rotate Image (90° clockwise, in-place) ---
def rotate(matrix):
    n = len(matrix)
    # Step 1: Transpose (swap across main diagonal)
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # Step 2: Reverse each row (mirror horizontally)
    for row in matrix:
        row.reverse()

# --- p054: Spiral Matrix (read in spiral order) ---
def spiral(matrix):
    result = []
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    while top <= bottom and left <= right:
        for c in range(left, right + 1): # → top row
            result.append(matrix[top][c])
        top += 1
        for r in range(top, bottom + 1): # ↓ right col
            result.append(matrix[r][right])
        right -= 1
        if top <= bottom: # GUARD: may be single row left
            for c in range(right, left - 1, -1): # ← bottom row
                result.append(matrix[bottom][c])
            bottom -= 1
        if left <= right: # GUARD: may be single col left
            for r in range(bottom, top - 1, -1): # ↑ left col
                result.append(matrix[r][left])
            left += 1
    return result
```

```
# --- p498: Diagonal Traverse (group by diagonal index) ---
def diagonal_traverse(mat):
    m, n = len(mat), len(mat[0])
    diags = {}
    for i in range(m):
        for j in range(n):
            d = i + j                # KEY: cells on same diagonal share i+j
            diags.setdefault(d, []).append(mat[i][j])
    result = []
    for d in range(m + n - 1):
        if d % 2 == 0:
            result.extend(reversed(diags[d])) # even diagonals go up-right → reverse
        else:
            result.extend(diags[d])          # odd diagonals go down-left → as-is
    return result

# --- p542: 01 Matrix (multi-source BFS from all 0-cells) ---
from collections import deque
def update_matrix(mat):
    m, n = len(mat), len(mat[0])
    dist = [[0] * n for _ in range(m)]
    queue = deque()
    for r in range(m):
        for c in range(n):
            if mat[r][c] == 0:
                queue.append((r, c))          # enqueue ALL sources first
            else:
                dist[r][c] = -1                # -1 = unvisited sentinel
    dirs = [(-1,0),(1,0),(0,-1),(0,1)]
    while queue:
        r, c = queue.popleft()
        for dr, dc in dirs:
            nr, nc = r + dr, c + dc
            if 0 <= nr < m and 0 <= nc < n and dist[nr][nc] == -1:
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))
    return dist
```

- **Complexity:**

- p048 Rotate: $O(n^2)$ time, **$O(1)$** space (pure in-place)
- p054 Spiral: $O(m \cdot n)$ time, $O(1)$ extra space (result not counted)
- p498 Diagonal: $O(m \cdot n)$ time, **$O(m \cdot n)$** space (diagonals dict)
- p542 01 Matrix: $O(m \cdot n)$ time, $O(m \cdot n)$ space (dist array + queue)

- **Killer gotchas:**

- **p054** — After consuming the top row (`top++`) and right col (`right--`), you **MUST** re-check `top <= bottom` before the bottom-row loop AND `left <= right` before the left-col loop. Without these guards a single remaining row or column gets appended twice.
- **p048** — Transpose only swaps $j > i$ pairs (`for j in range(i+1, n)`); starting from $j=0$ would double-swap and undo the transpose.
- **p498** — The grouping approach ($d = i+j$) is simpler and safer than direction-flip simulation. Even-indexed diagonals ($d \% 2 == 0$) go **up-right** in the output, so you reverse the collected

list; odd diagonals go down-left as-is.

- **p542** — Use `dist[nr][nc] == -1` (not a `visited` set) as the unvisited sentinel. This works because distance is always non-negative, so `-1` is unambiguous. Enqueue ALL 0-cells before starting BFS — not one at a time.

Problem	Approach	Key Trick
p048 Rotate Image	Transpose then reverse each row	Transpose swaps (i, j) (j, i) for $j > i$ only; then <code>row.reverse()</code> = clockwise 90°
p054 Spiral Matrix	4 boundary pointers, shrink inward	Guard <code>top bottom / left right</code> before bottom-row & left-col loops to avoid double-append
p498 Diagonal Traverse	Group cells by $d=i+j$; reverse even d	All cells with same $i+j$ lie on one diagonal; even $d \rightarrow$ up-right $\rightarrow$ reverse the list
p542 01 Matrix	Multi-source BFS from all 0-cells	Seed queue with every 0, mark 1-cells as <code>dist=-1</code> ; BFS propagates distance outward

monotonic_stack

- **Mindset:** Keep a stack of **indices** whose values are strictly monotonic (increasing or decreasing). When a new element breaks this monotonicity, it acts as the right-boundary constraint for the elements popped from the stack.
- **When to use:** “next greater/smaller”, “distance to next warmer/colder”, “largest rectangle bounded by smaller neighbors”, “contribution of each element as min/max of all subarrays”, “132 subsequence search”.
- **Skeleton (Next Greater / Decreasing Stack — p739 / p503):**

```
def next_greater(nums: list[int]) -> list[int]:
    n = len(nums)
    answer = [-1] * n
    stack = []  # stores indices
    for i in range(2 * n):  # 2 * n loops for circular array (p503)
        idx = i % n
        while stack and nums[stack[-1]] < nums[idx]:
            answer[stack.pop()] = nums[idx]
        if i < n:  # only push indices in first pass (p503)
            stack.append(idx)
    return answer
```

- **Skeleton (Largest Rectangle / Increasing Stack with Sentinel — p084):**

```
def largest_rectangle_area(heights: list[int]) -> int:
    stack = []
    max_area = 0
    heights = heights + [0]  # sentinel 0 forces a flush of all remaining elements
    for i, h in enumerate(heights):
        while stack and h < heights[stack[-1]]:
            height = heights[stack.pop()]
```

```
        # Left boundary is the new top of stack, right boundary is i
        width = i if not stack else i - stack[-1] - 1
        max_area = max(max_area, height * width)
    stack.append(i)
return max_area
```

- **Skeleton (Subarray Min Contribution — p907):**

```
def sum_subarray_mins(arr: list[int]) -> int:
    n = len(arr)
    left = [-1] * n
    right = [n] * n
    stack = []

    # Pass 1: find first smaller to left (strictly smaller)
    for i in range(n):
        while stack and arr[stack[-1]] >= arr[i]:
            stack.pop()
        left[i] = stack[-1] if stack else -1
        stack.append(i)

    stack.clear()

    # Pass 2: find first smaller or equal to right (prevents double-counting duplicates)
    for i in range(n - 1, -1, -1):
        while stack and arr[stack[-1]] > arr[i]:
            stack.pop()
        right[i] = stack[-1] if stack else n
        stack.append(i)

    # sum up: (i - left[i]) * (right[i] - i) is the number of subarrays where arr[i] is the
    return sum(arr[i] * (i - left[i]) * (right[i] - i) for i in range(n)) % (10**9 + 7)
```

- **Complexity:** $O(N)$ time (each index is pushed and popped at most once). $O(N)$ space for stack and boundary arrays.
- **Killer gotchas:**

1. **Rectangle width index math (p084)** — width is `i - stack[-1] - 1` if stack is not empty, else `i`. It is NOT `i - popped_idx`. The left boundary is defined by the element that remains at the top of the stack, not the one that was popped.
2. **Sentinel appending (p084)** — you must append 0 (or -1 for positive values) at the end of heights. Without it, elements left in the stack at the end of the loop never get popped and processed.
3. **Duplicate boundaries in contribution sum (p907)** — to avoid double counting elements of equal values in subarrays, one search pass must use strictly greater-than/strictly less-than (`>=` / `>`) while the other uses non-strict inequality (`>` / `>=`).
4. **132 Pattern (p456) backward traversal** — traversal must go from right-to-left. Keep stack decreasing. `third` represents the `nums[k]` candidate. If we see `nums[i] < third`, return True (since `third` was popped by some larger `nums[j]` on stack, we have `nums[i] < third < nums[j]`).
5. **Car fleet (p853) sorted positions** — sorting positions descending makes it a monotonic sequence of arrival times from front to back. A car behind forms a new fleet only if its time to target is strictly greater than the current slowest time.

Problem	Essence	Key trick
p739 Daily Temperatures	Next warmer day distance	Decreasing stack of indices; <code>ans[j] = i - j</code> on pop
p084 Largest Rectangle	Max rectangle in histogram	Increasing stack + sentinel 0; <code>width = i - stack[-1] - 1</code> on pop
p503 Next Greater II	Circular next greater element	Loop $2*n$ with <code>idx = i % n</code> ; only push to stack when <code>i < n</code>
p456 132 Pattern	Check subsequence <code>nums[i] < nums[k] < nums[j]</code>	Loop backwards; stack tracks <code>j</code> (max), <code>third</code> tracks popped <code>k</code>
p853 Car Fleet	Number of fleets reaching target	Sort pos descending; count times where <code>time > slowest_time</code>
p907 Sum of Subarray Mins	Subarray minimums contribution	Two passes (left < and right bounds); <code>contribution = val * left_gap * right_gap</code>

union_find

- **Mindset:** Track dynamic connectivity (“same set?”) with near- $O(1)$ `union/find` via path compression + union by rank — incremental grouping, faster than BFS/DFS for repeated undirected “who’s connected” queries. Think: forest of roots where `find` returns the root and `union` merges two trees.
- **When to use:** **Undirected** connectivity only: component count, redundant/cycle edge detection, equality satisfiability, dynamic grouping. NOT for directed graphs or shortest path.
- **Skeleton:**

```
class UnionFind:
    def __init__(self, n: int):
        self.parent = list(range(n))    # each node is its own root
        self.rank   = [0] * n

    def find(self, x: int) -> int:
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])    # path compression (recursive)
        return self.parent[x]

    def union(self, x: int, y: int) -> bool:
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False    # already same set - cycle detected
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx    # always attach smaller rank under larger
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1
        return True    # successful merge

# p323 driver
uf = UnionFind(n)
```

```

components = n
for u, v in edges:
    if uf.union(u, v):
        components -= 1          # one fewer component per merge
return components

# p684 driver - nodes are 1-indexed! parent size = n+1
parent = list(range(len(edges) + 1)) # NOT range(len(edges))
rank   = [0] * (len(edges) + 1)
for u, v in edges:
    if not union(u, v):
        return [u, v]          # first edge that fails = redundant

# p990 driver - 26-node fixed pool; char → int via ord(c) - ord('a')
parent = list(range(26))
rank   = [0] * 26
# Pass 1: union all "==" (eq[1]=='=' because format is "a==b")
for eq in equations:
    if eq[1] == '=':
        union(ord(eq[0])-ord('a'), ord(eq[3])-ord('a'))
# Pass 2: reject any "!=" where both sides share a root
for eq in equations:
    if eq[1] == '!=':
        if find(ord(eq[0])-ord('a')) == find(ord(eq[3])-ord('a')):
            return False
return True

```

- **Complexity:** $O(E \cdot (N))$ $O(E)$ time (= inverse Ackermann, effectively constant). $O(N)$ space for parent + rank arrays.
- **Killer gotchas:**
 1. **p990 MUST be two passes** — union every == *first*, only then check !=. Interleaving fails: a later == can merge sets that an earlier != wrongly passed.
 2. **p684: 1-indexed nodes** — edges use nodes 1..n, so parent must be size n+1 (list(range(len(edges)+1))). Off-by-one here causes silent wrong answers.
 3. **p684: stop at the first failed union** — problem guarantees exactly one redundant edge; the answer is the last such edge in input order, which is the *first union* failure.
 4. **Path compression is recursive** — Python default recursion limit (~1000) is fine for constraints here (n ≤ 2000), but consider iterative find for very large n.
 5. **Rank only increases when both sides have equal rank** — a common mistake is always incrementing rank[rx].

Problem	Essence	Key trick
p323 Connected Components	n - successful_unions	Decrement counter only when union() returns True
p684 Redundant Connection	First edge linking already-connected nodes	Nodes are 1-indexed ; parent size = n+1; return on first False
p990 Equality Equations	Union all ==, then reject != contradictions	Fixed 26-node pool; eq[1]=='=' to detect ==; eq[0]/eq[3] are the variable chars

Greedy vs DP

- **Greedy** — one forward pass + one state variable suffices and the optimal local choice never forecloses a better global one (p055, p134, p452, p455).
- Reach for **DP / multi-pass** when a position's answer is constrained by **both sides at once** (p135 candy: $L \rightarrow R$ and $R \rightarrow L$ then max) or when choices interact.

Graph: BFS vs Topological Sort vs Union-Find

- **Linear order / dependencies / cycle in a *directed* graph** $\rightarrow$ **topological sort** (Kahn's) (p207, p210).
- **Distance / shortest path / nearest-X / flood fill** $\rightarrow$ **BFS/DFS** (multi-source in p542).
- **"Same component?" with *undirected* edges, possibly streaming** $\rightarrow$ **union-find** (p323, p684, p990) — incremental, avoids full traversal.
- Just need **in/out-degree of one special node** $\rightarrow$ neither; pure **degree array** (p997).

Binary Search: plain vs modified vs answer-space

- **plain** — compare `nums[mid]` to **target** in a cleanly sorted array.
- **modified (rotated)** — discover *which half is sorted* before deciding where target lives.
- **answer-space** — abandon the array; search a numeric **answer** via a monotonic **feasible(mid)** predicate ("minimize the max"). Loop **while** `lo < hi`, move `hi = mid` on success.

Backtracking vs Subsets

- Identical choose $\rightarrow$ explore $\rightarrow$ un-choose skeleton, opposite philosophy.
- **Subsets** records **every** partial path (the answer-set *is* the path-space); rarely prunes.
- **Backtracking** records only **goal-reaching** paths (**remaining** == 0, square complete); prunes hard. Subsets = *enumerate all*; backtracking = *search for the valid*.

Heap direction (top-k)

- A **min-heap** of size **k** keeps the **k LARGEST** (you evict the small ones) — backwards from intuition. To keep **k** smallest, negate keys.